CSCI3230 (ESTR3108)
Fundamentals of Artificial Intelligence

Lecture 8. Transformers

Prof. Qi Dou

Email: qidou@cuhk.edu.hk
Office: Room 911, 9/F, SHB

Dept. of Computer Science & Engineering
The Chinese University of Hong Kong



Outline

Part 1. Basics

Part 2. Vision Transformers

Part 3. Variants of Transformers

Part 4. Summary

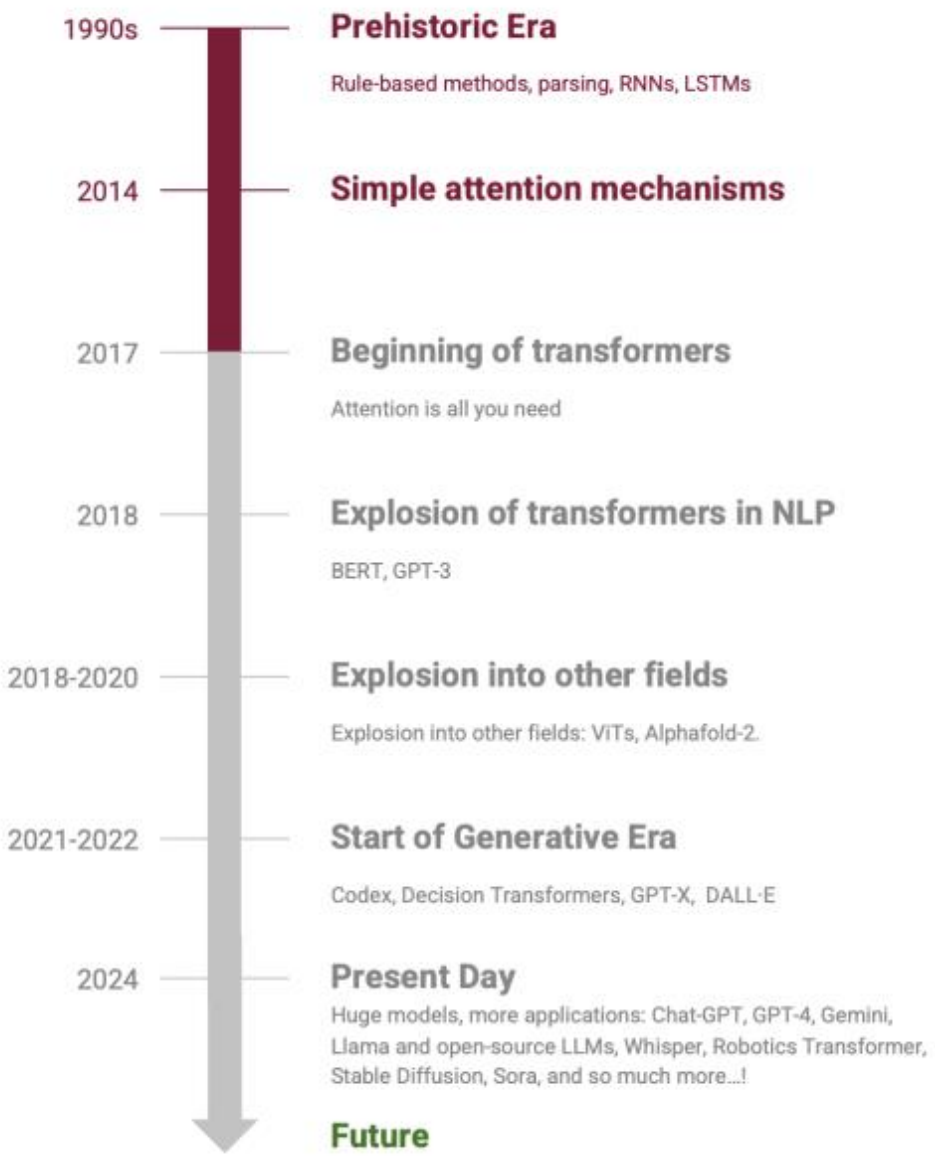


Part 1. Basics

Basic Concepts

- The Transformer is a neural network architecture initially designed for **sequential modeling** in natural language processing (NLP), and now applied to fields like vision and audio.

- Structurally, a Transformer is composed of linear layers, normalization layers and activation functions.



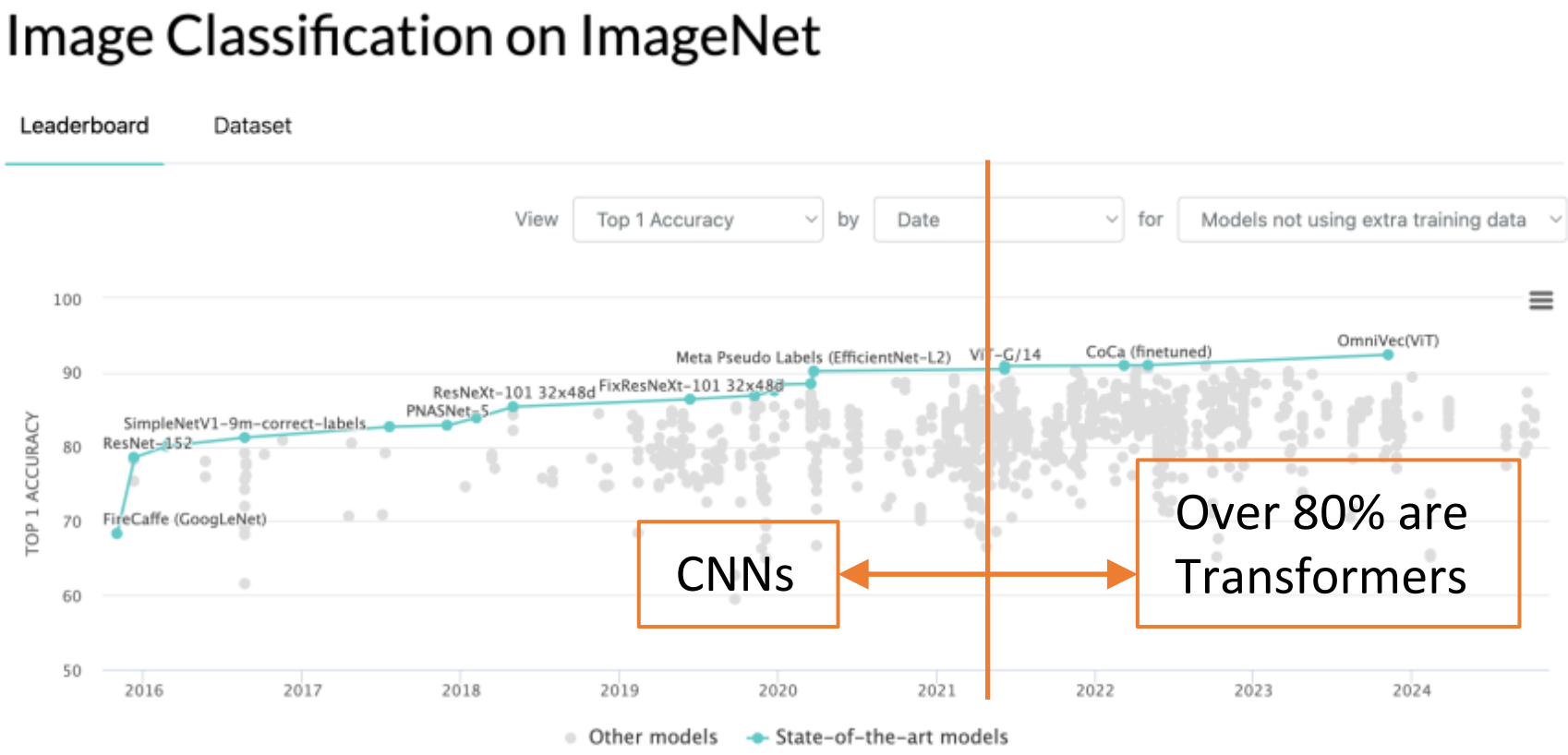
Basic Concepts

- In NLP, RNN (recurrent neural network) based sequence to sequence models was leading. Transformer has been the mainstream architecture since 2017.



Basic Concepts

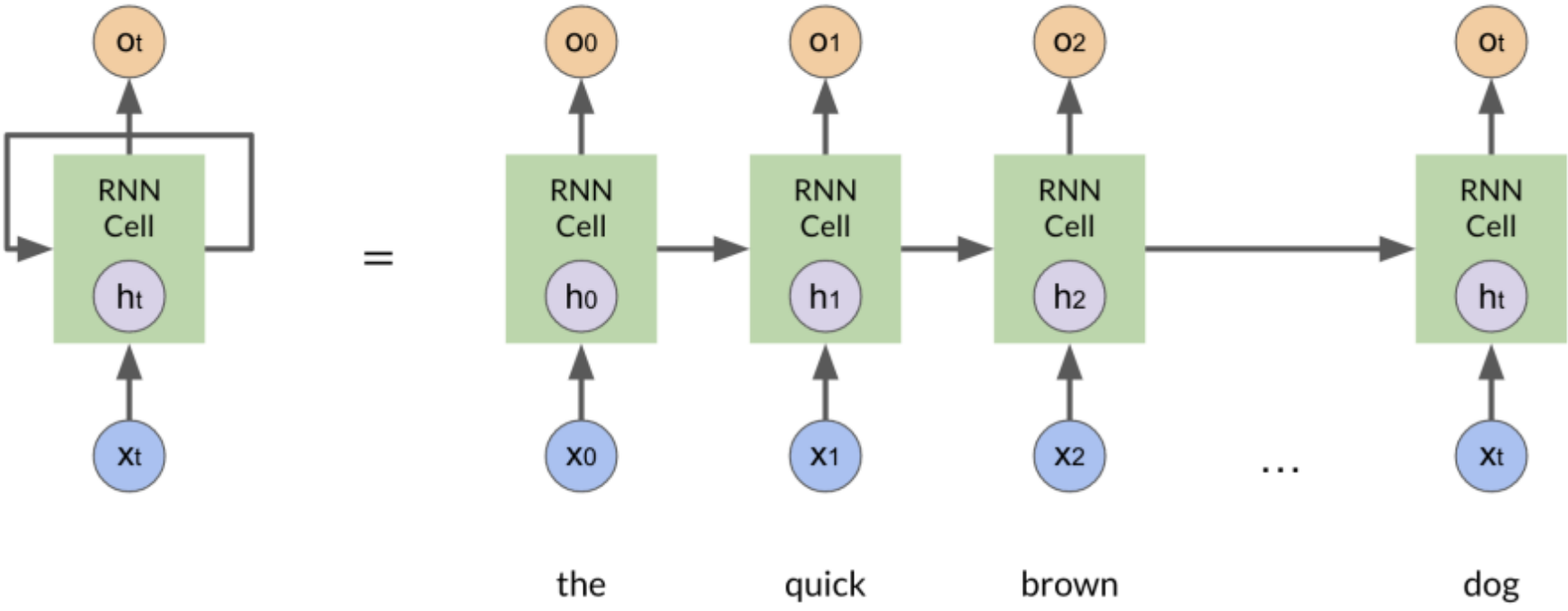
- In computer vision (CV), for a long time, the dominant architecture was CNN. However, Transformers are now being embraced in computer vision.



Online leaderboard: https://paperswithcode.com/sota/image-classification-on-imagenet

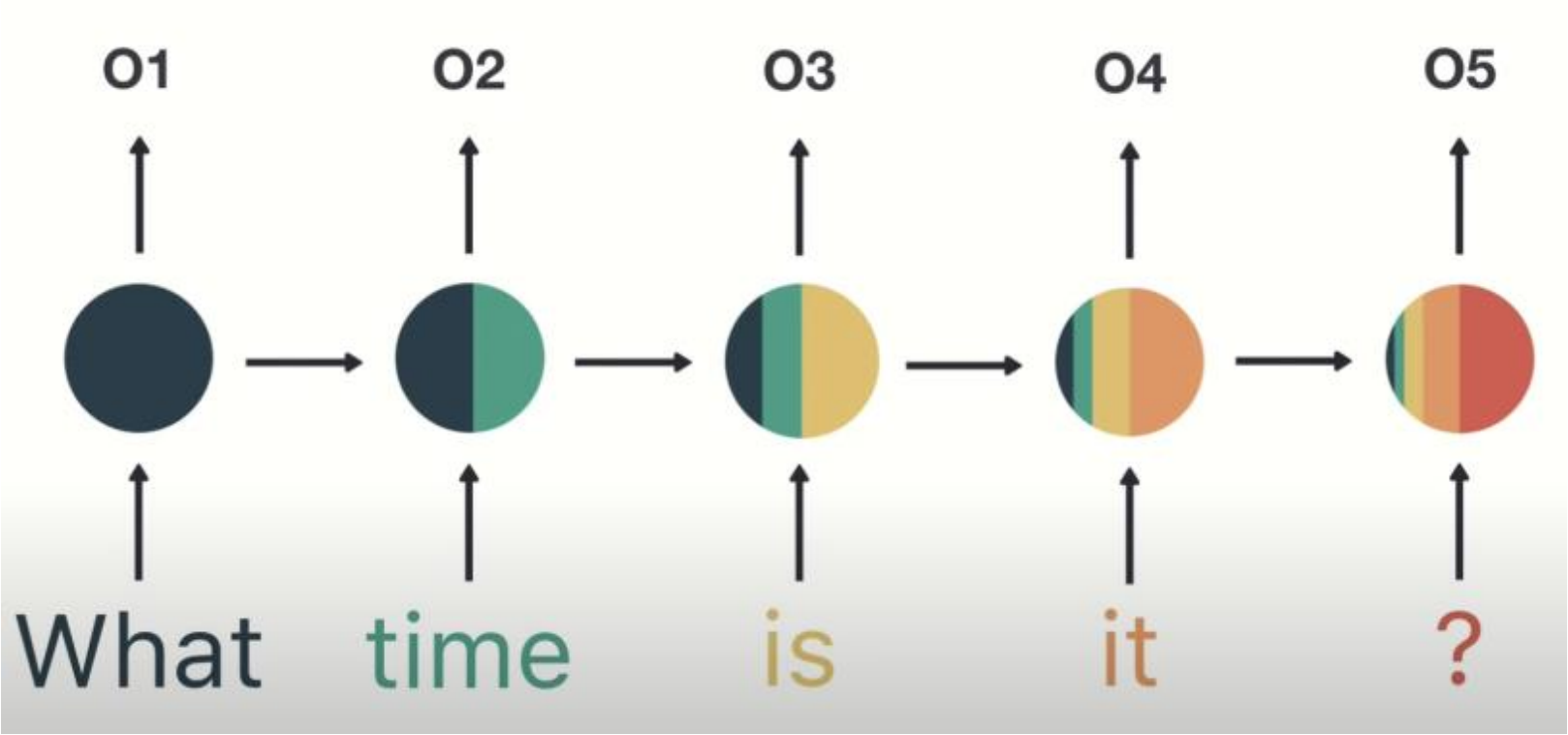
Sequential Modeling - Recurrence

- Before Transformer, Recurrent Neural Network (RNN) is the first choice for sequential modeling
- Given a sequence of inputs (e.g., a sentence), RNN works in a recurrence manner



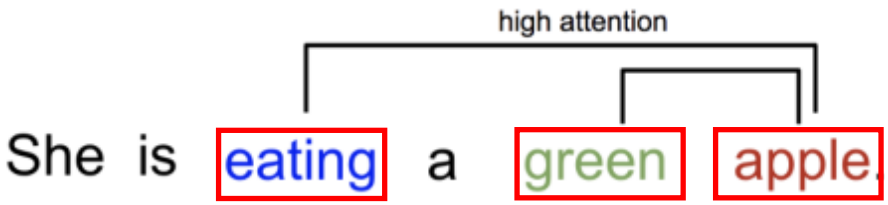
Sequential Modeling - Recurrence

- However, RNNs struggle with extremely long sequence inputs
  - Forgetting occurs in the recurrence
  - The inherently recurrent nature precludes parallelization in computation



Sequential Modeling - Attention

- Transformers eschew recurrence and instead rely entirely on an **attention mechanism**.
- Intuition: Though the input sequence could be long, only a small subset of the sequence might be related
- For example, only a few words in the sentence are relevant to answering the following question.

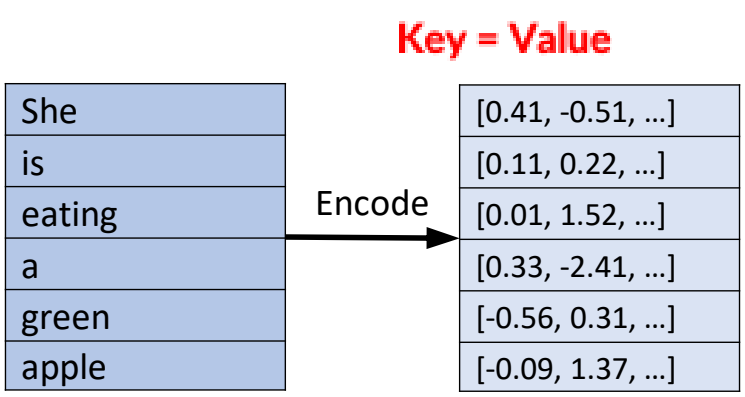


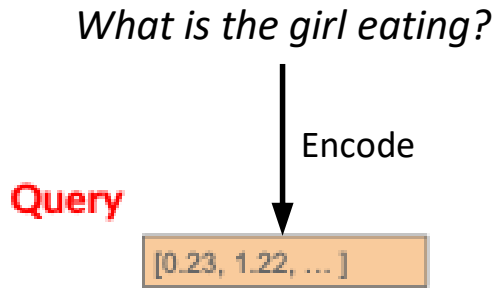
Query: What is the girl eating?

Sequential Modeling - Attention

- The attention mechanism is based on three types of vectors as a retrieval system: **query, key, value**.

- For example, when you search for videos on Youtube, the search engine will map your **query** (text in the search bar) against a set of **keys** (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (**values**).

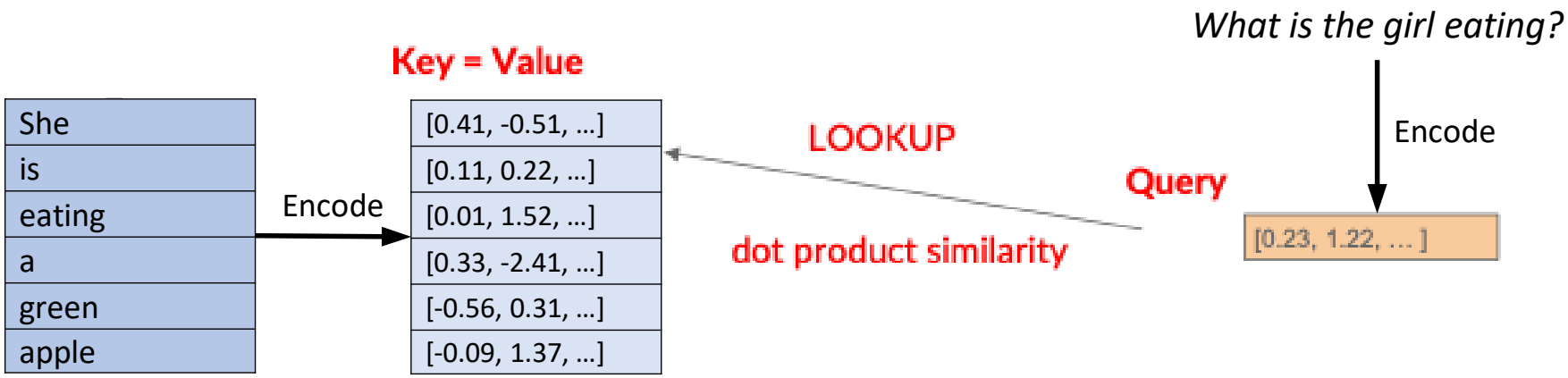




Sequential Modeling - Attention

- The attention mechanism operates on a find-then-retrieve paradigm.

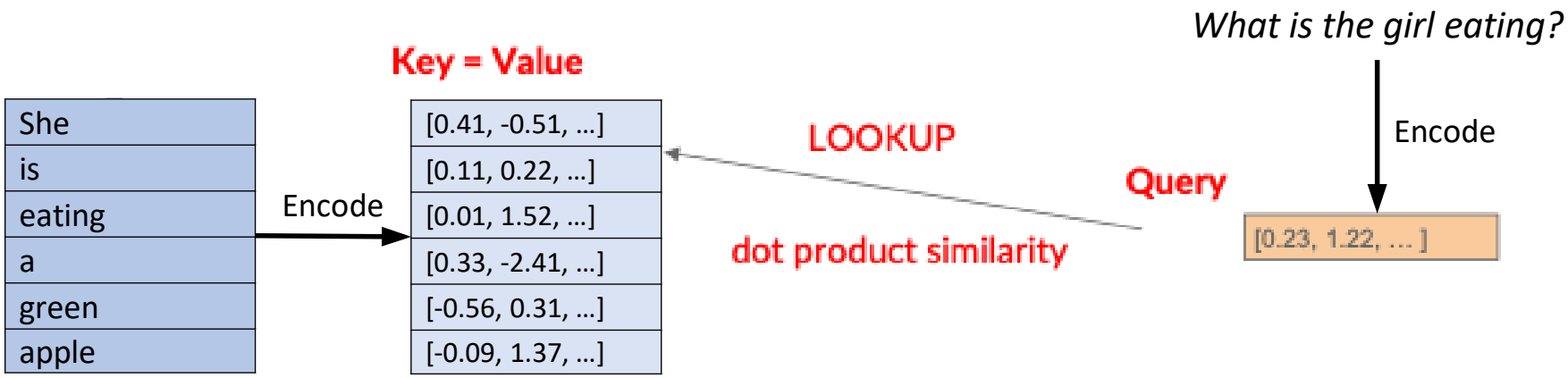
Step 1: similarity calculation



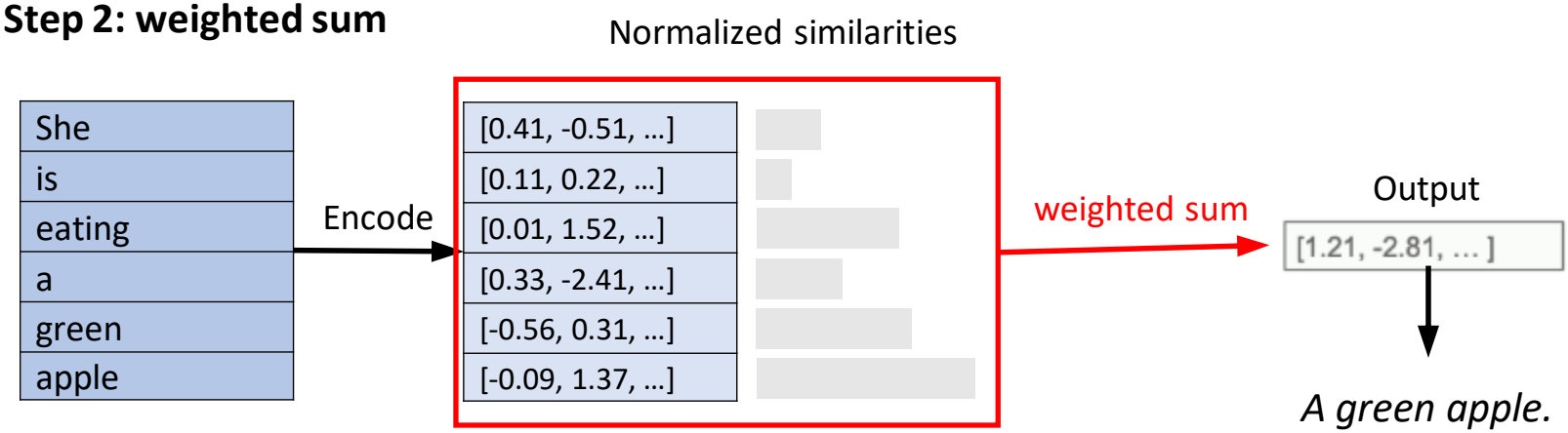
Sequential Modeling - Attention

- The attention mechanism operates on a find-then-retrieve paradigm.

Step 1: similarity calculation



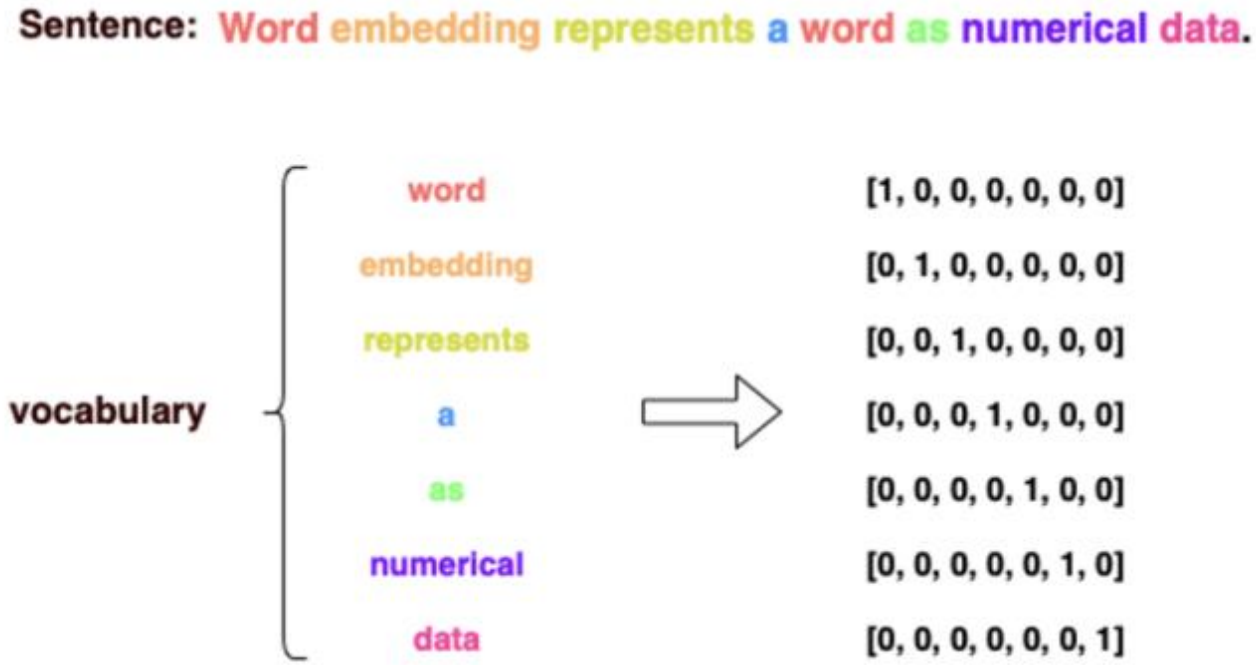
Step 2: weighted sum



Tokenization

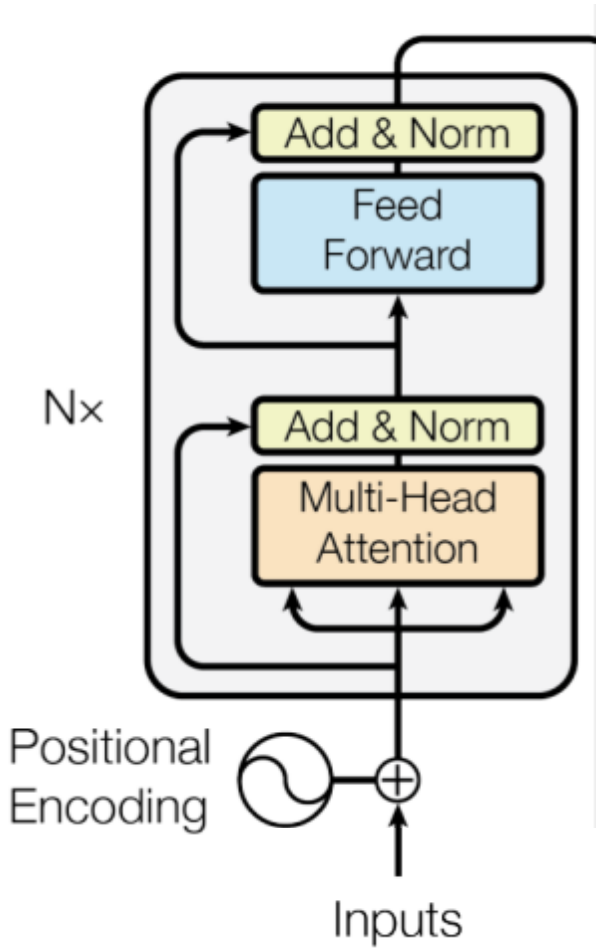
- Before feeding into Transformer, the input sentence should be first converted into a sequence of **tokens**
  - Each token is an 1D vector
  - The initial tokens are obtained by looking up a dictionary that stores the mapping between words and tokens

Example of one-hot tokenization



Transformer Architecture

- Repeated Transformer Blocks
  - Multi-Head Attention (key module)
  - Feed Forward
  - Residual Addition and Normalization
- Positional Encoding



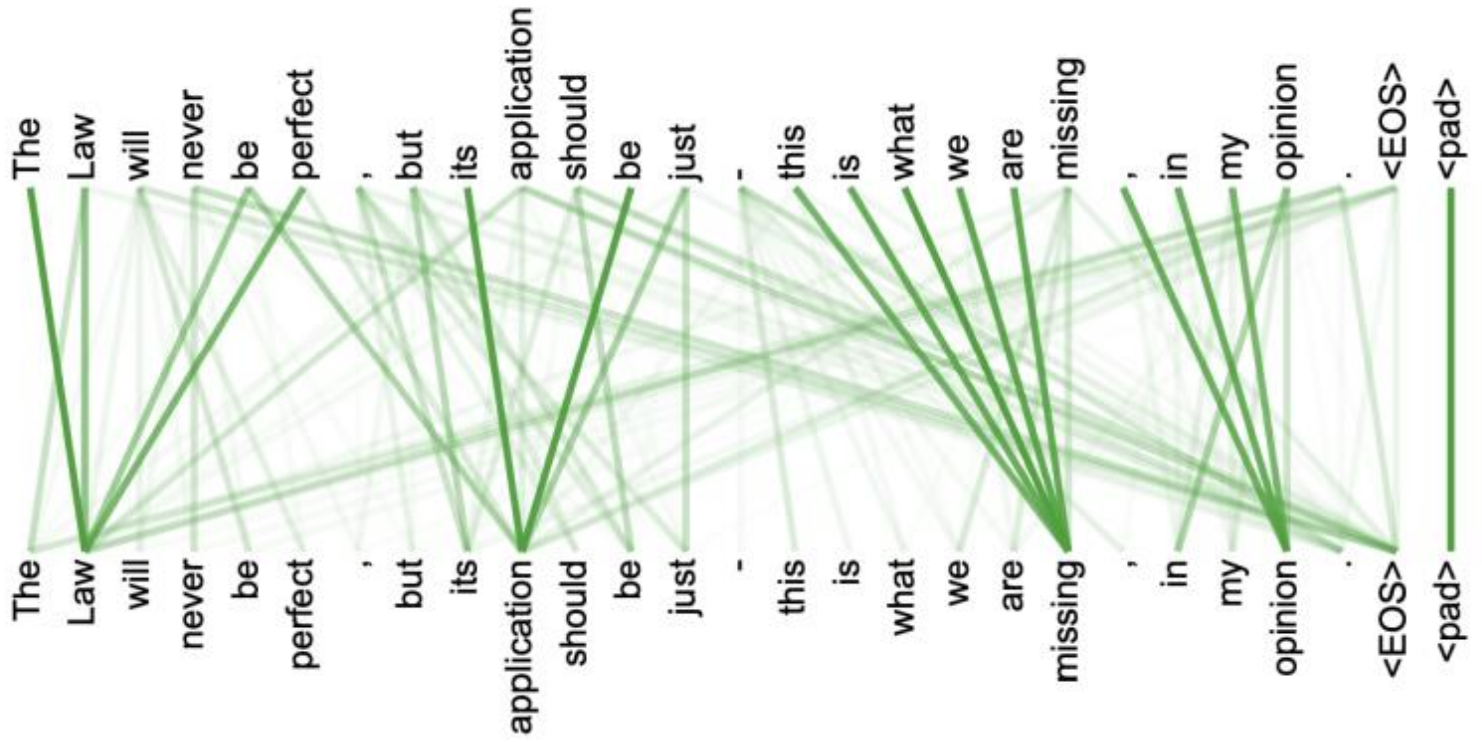
Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Vaswani, A. "Attention is all you need." Advances in Neural Information Processing Systems (2017).

Self-Attention

- Self-attention involves tokens in the input sequence attending to other tokens within the same input.
- Self-attention works as a feature encoding process, focusing on relevant information and capturing long-range dependencies.



Self-Attention

- Assume we have a sequence of 3 tokens (e.g., a sentence of 3 words)
- Now we simulate the forward process for token 1 in self-attention



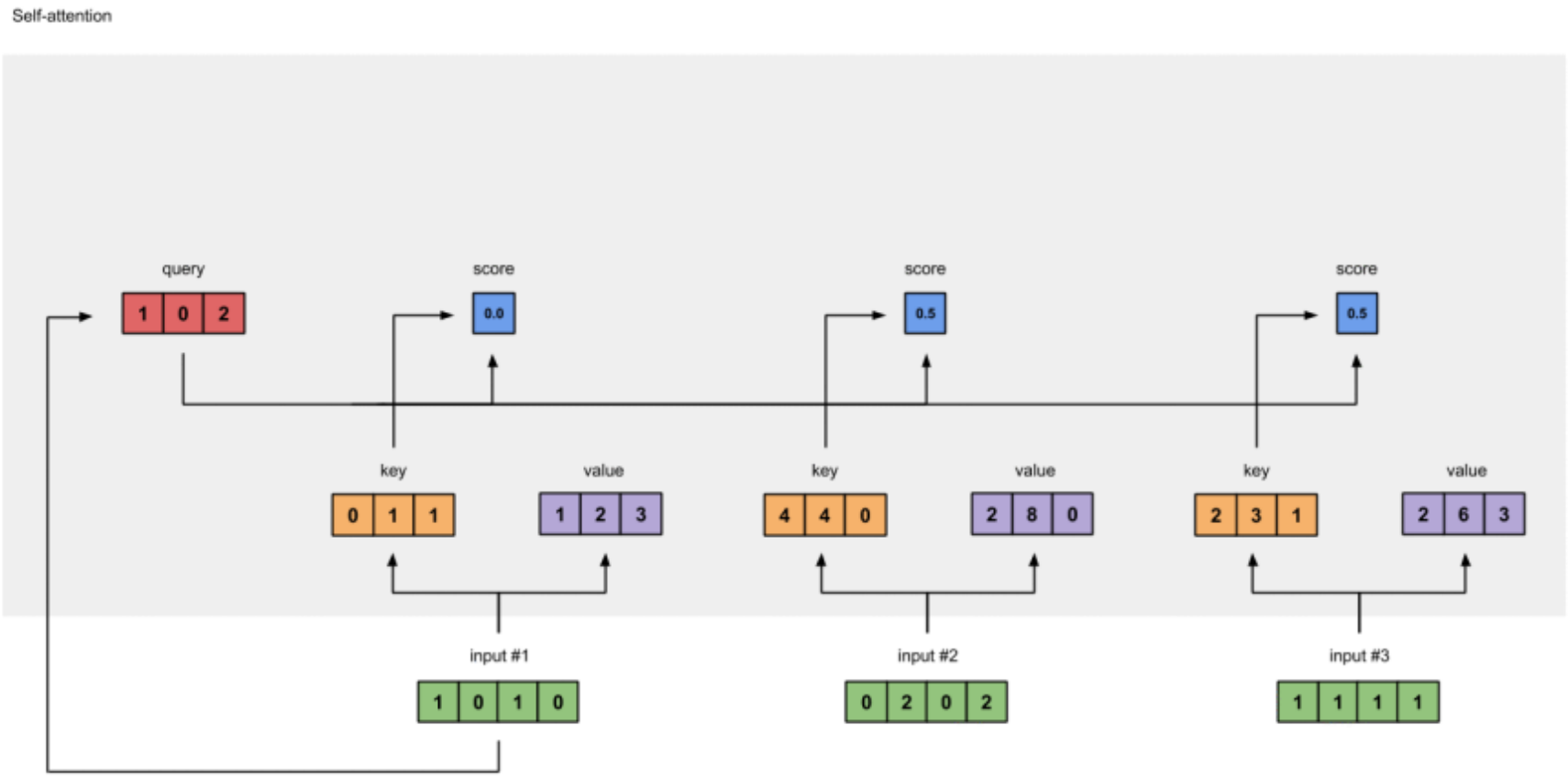
Self-Attention

- Give each token a KEY, QUERY, and VALUE representation
  - These KEY, QUERY, and VALUE representations are just linear projections of the initial token representations
  - We use three independent linear layers to generate KEY, QUERY and VALUE representations



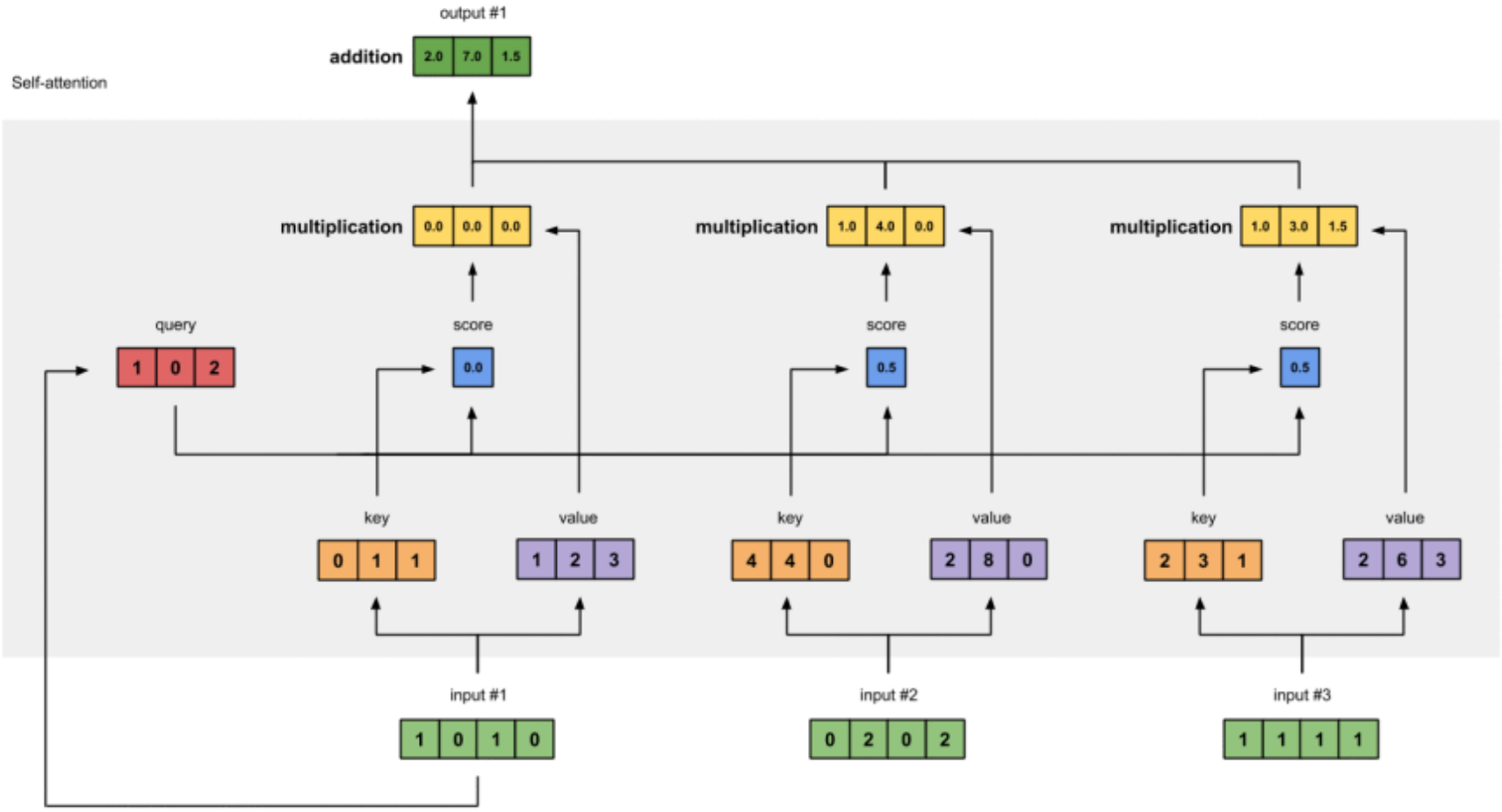
Self-Attention

- We use its QUERY to find the most relevant token(s) in the sequence based on their respective KEY(s)
  - Calculate similarity with dot product
  - Calculate softmax distribution based on dot product values



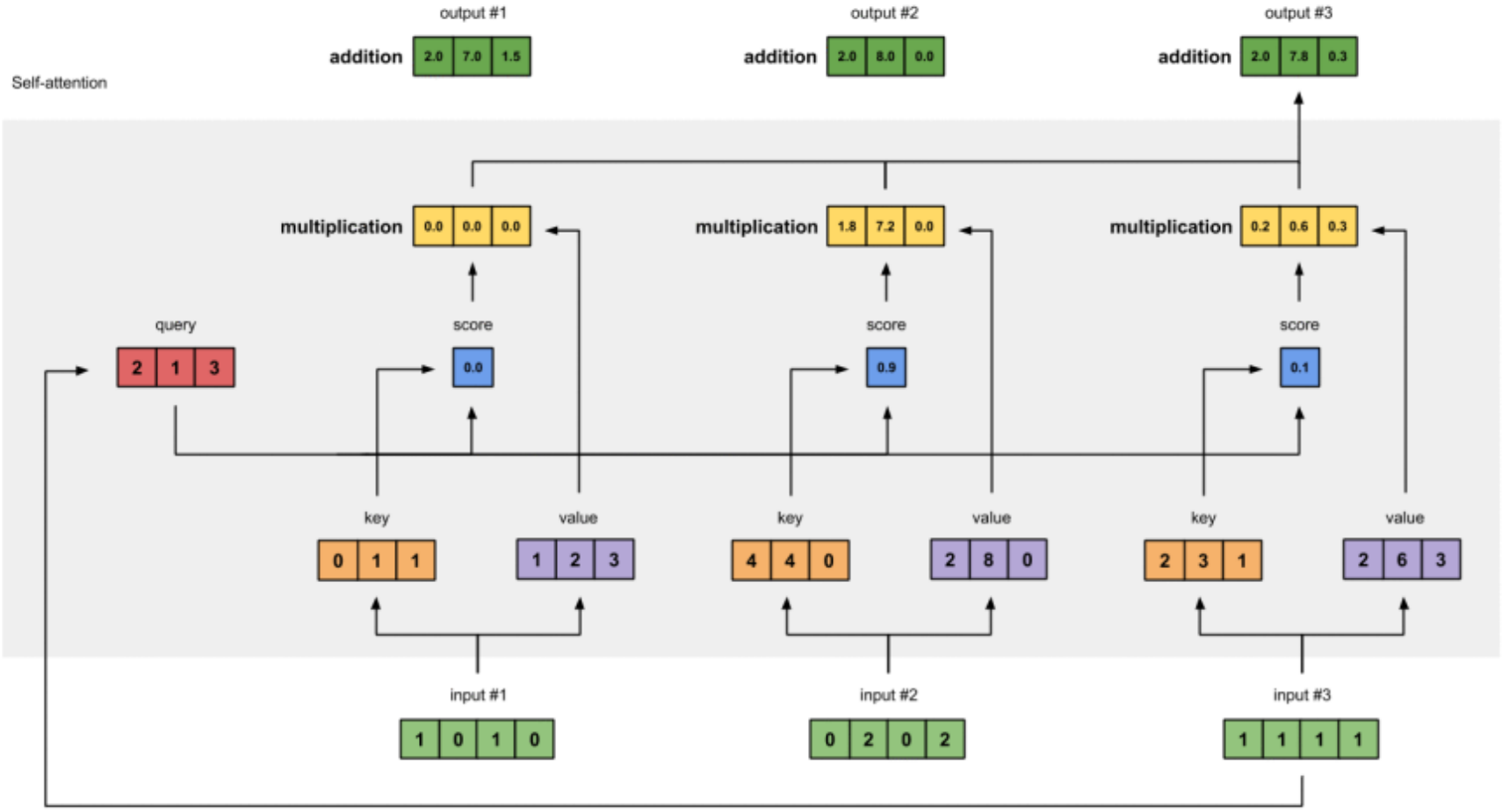
Self-Attention

- We combine the VALUE(s) of the relevant token(s) using weighted sum based on similarities
- Then we output the new representations of that token



Self-Attention

- Apply the same process for all tokens
- Note that the processes for all tokens are performed in parallel using matrix calculations



Self-Attention

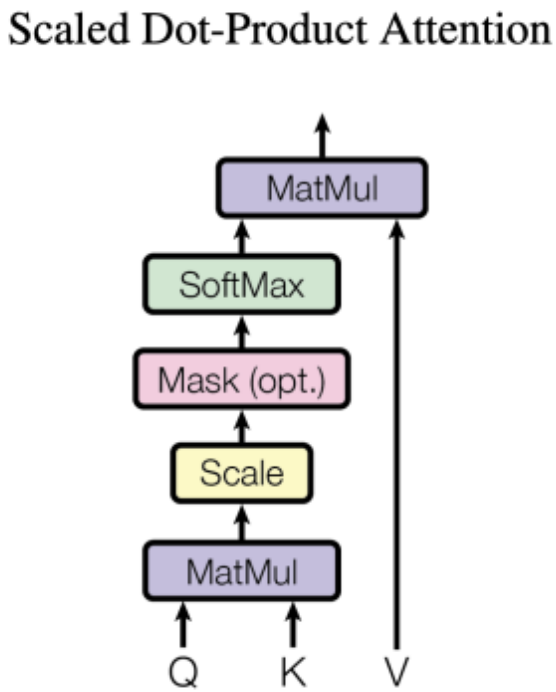
- In practice, we scale dot product similarities by a factor of $\frac{1}{\sqrt{d_k}}$, where d_k is the dimension of KEY representations.
- In higher dimensions, if dot product values increase, causing some components of softmax to have extremely small gradients.

Self-Attention

- In practice, we scale dot product similarities by a factor of $\frac{1}{\sqrt{d_k}}$, where d_k is the dimension of KEY representations.
- In higher dimensions, if dot product values increase, causing some components of softmax to have extremely small gradients.

- The attention process is formulated as

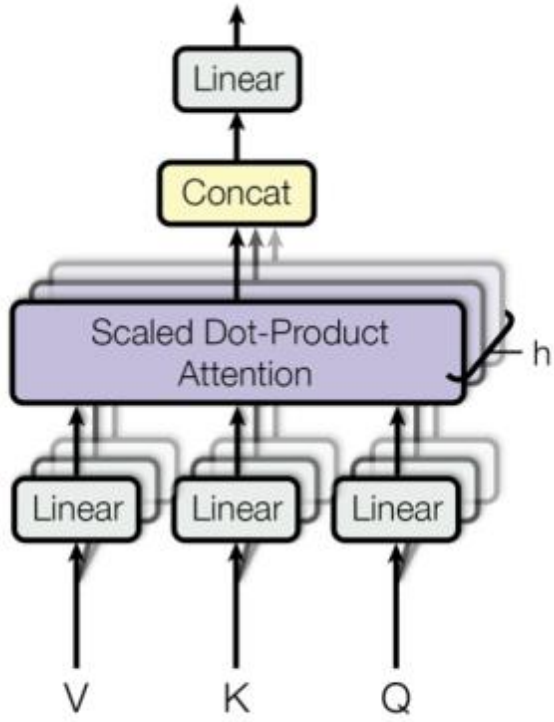
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Multi-Head Attention

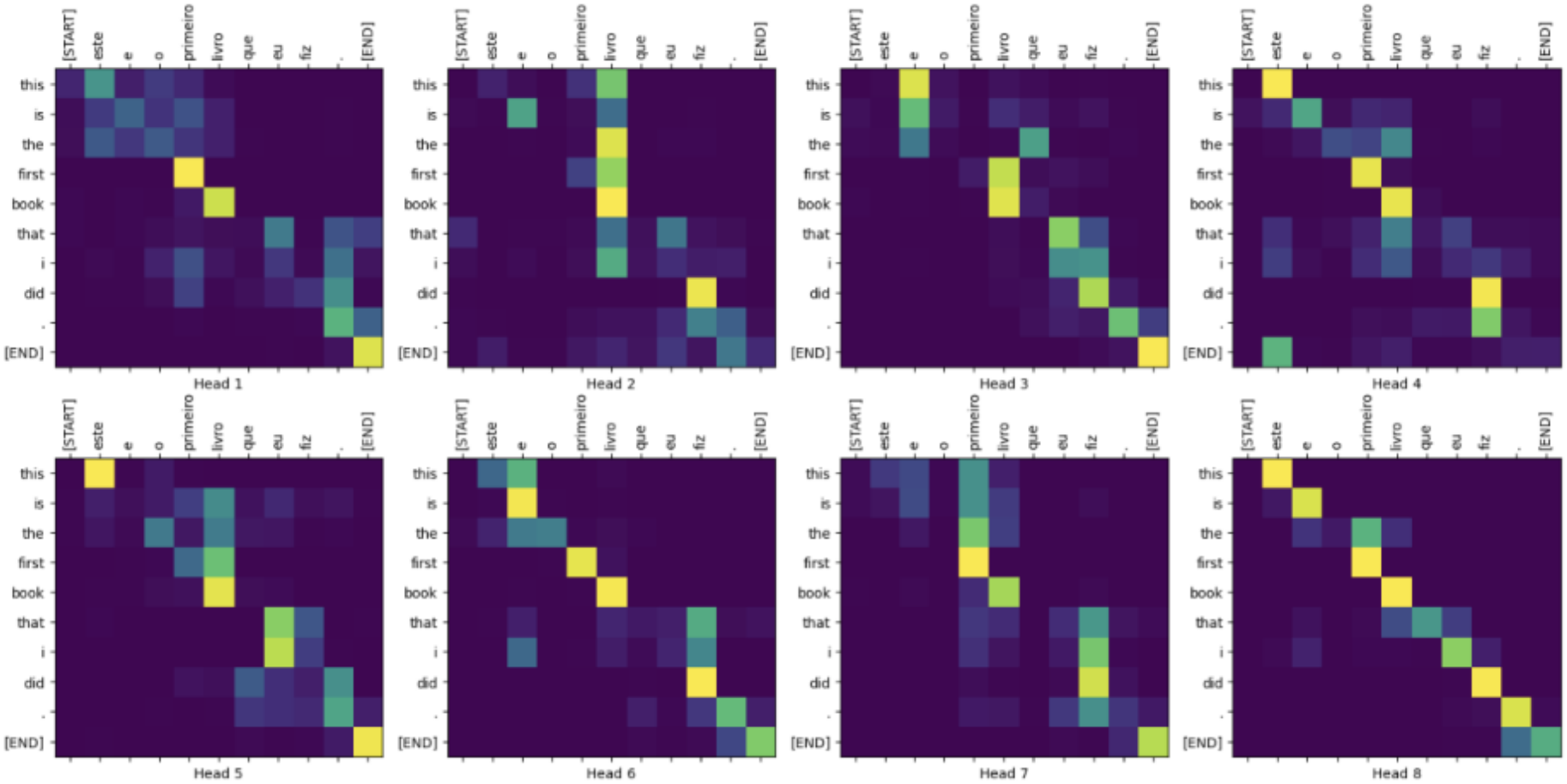
- Project each (K, Q, V) triplet down h different times with different sets of projection weights
  - h is the number of "heads"
  - Process each "head" independently in parallel, re-concatenate all outputs and re-project them to get the final embedding

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Multi-Head Attention

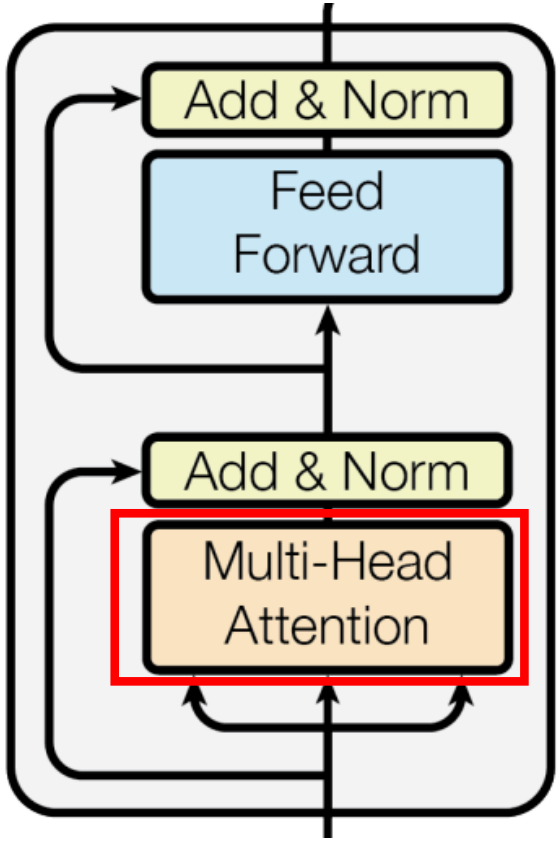
- Each head captures different semantics



Visualization of attention from different heads

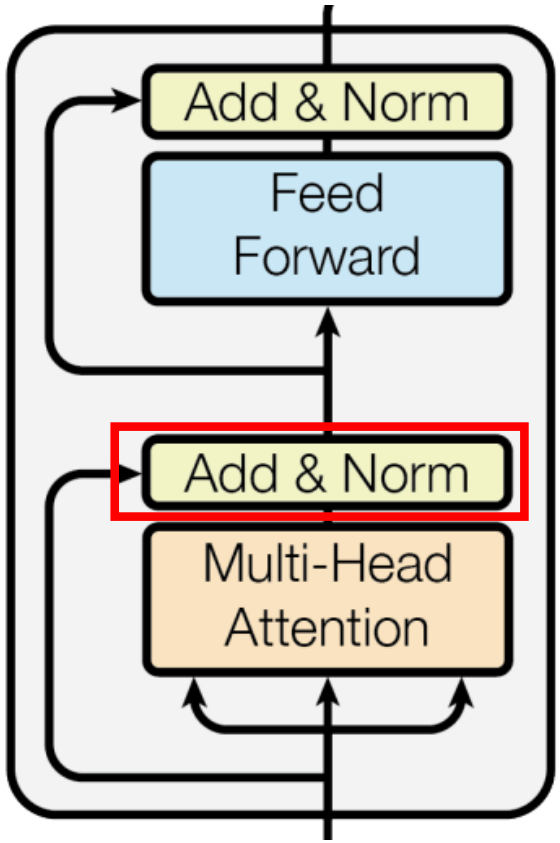
Multi-Head Attention

- Multiple heads provide a way to model different representations of the same token (as it should be able to attend to a different set of tokens) and consider all of them together



Add & Norm

- Residual connections (Add)
  - To remedy vanishing gradients in models with lots of layers
  - Think back to the motivation for ResNets
- Layer normalization (Norm)



Feed Forward

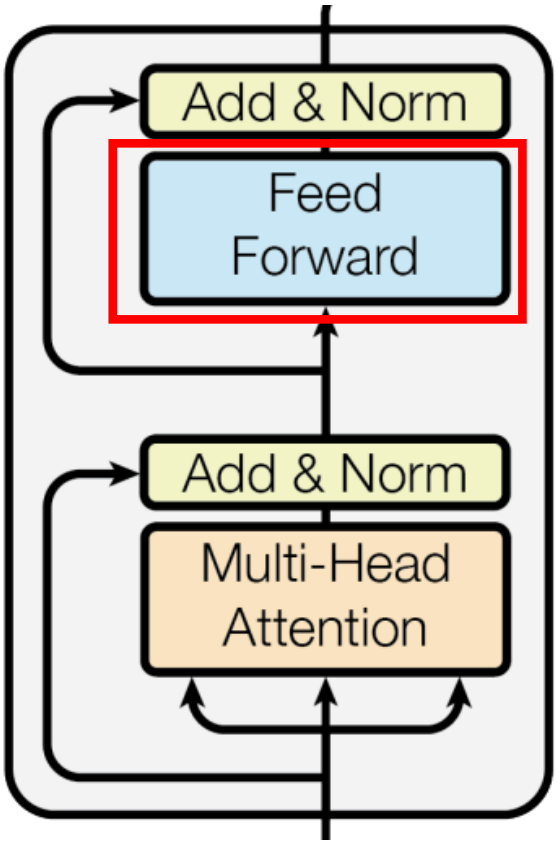
- Self-attention models relationships between tokens, but there is no element-wise non-linearity

Feed Forward

- Feed Forward layers are usually two-layer MLPs (weights shared w.r.t. position) applied to each token embedding individually

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Attention is “recombining” input vectors, which is linear operation
  - MLP adds non-linearities
  - MLP adds linear layers to “mix” channel values



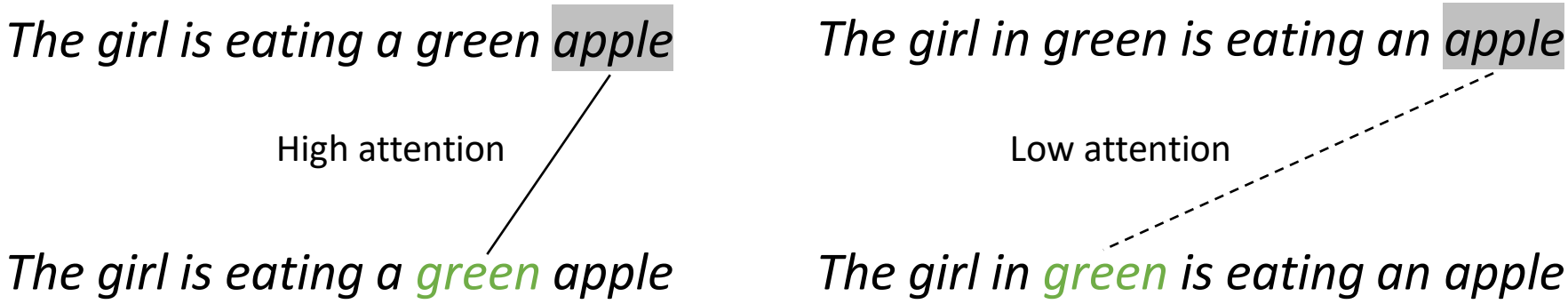
Positional Encoding (PE)

- Transformer blocks do not inherently account for position. Self-attention process tokens based on the similarities only.

Positional Encoding (PE)

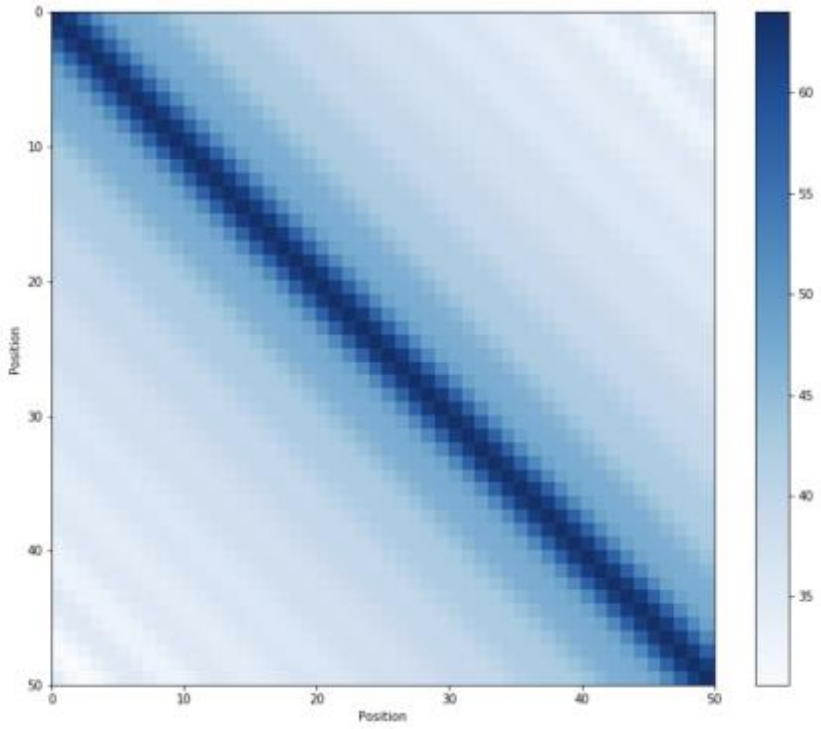
- Transformer blocks do not inherently account for position. Self-attention process tokens based on the similarities only.

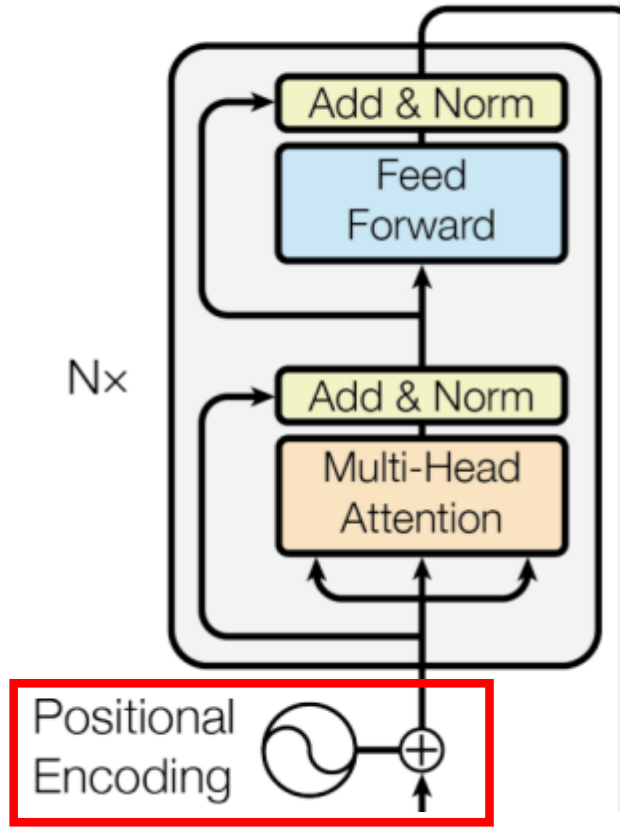
- However, the position of tokens in a sequence also matters.
  - Closer tokens should share greater similarities



Positional Encoding (PE)

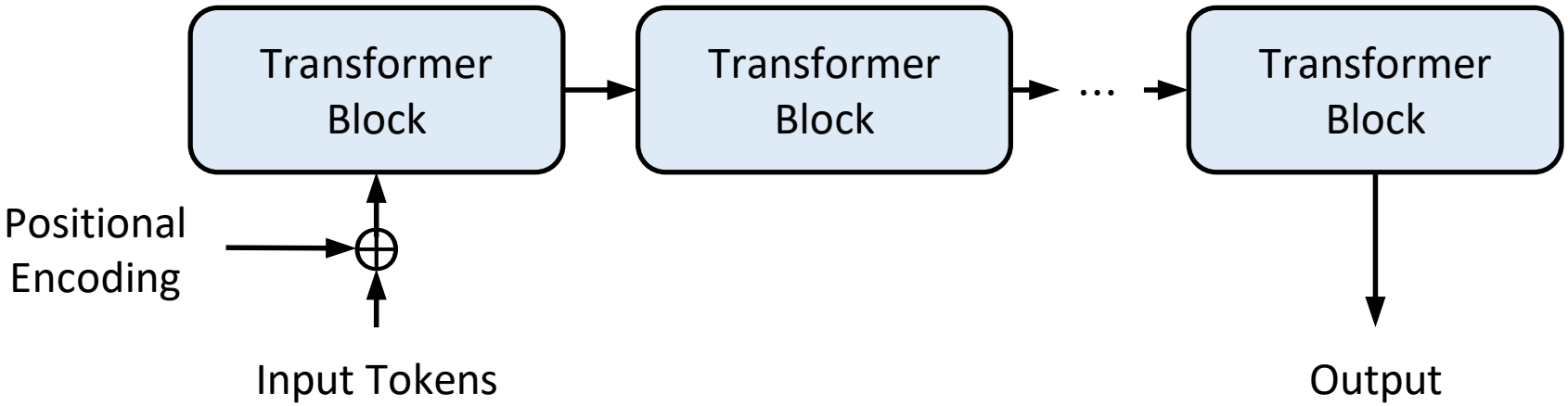
- Solution: Add positional embeddings to each token based on its position.
  - PE are vectors with the same dimensions as the tokens
- PE of closer tokens have higher dot-product similarities.





Stacked Transformer Block

- The Transformer is made up of stacked transformer blocks
  - The Transformer layer process input tokens and output the deeper feature embeddings
  - With model going deeper, the word tokens are getting more representative as well





Part 2. Vision Transformers

Spatial Attention

- Intuition: For 2D images, the attention mechanism naturally occurs when interpolating images
  - Given a query, we pay attention to relevant areas
  - The query could be (a part of) the image itself (i.e., self-attention)



A large white bird standing in a forest.



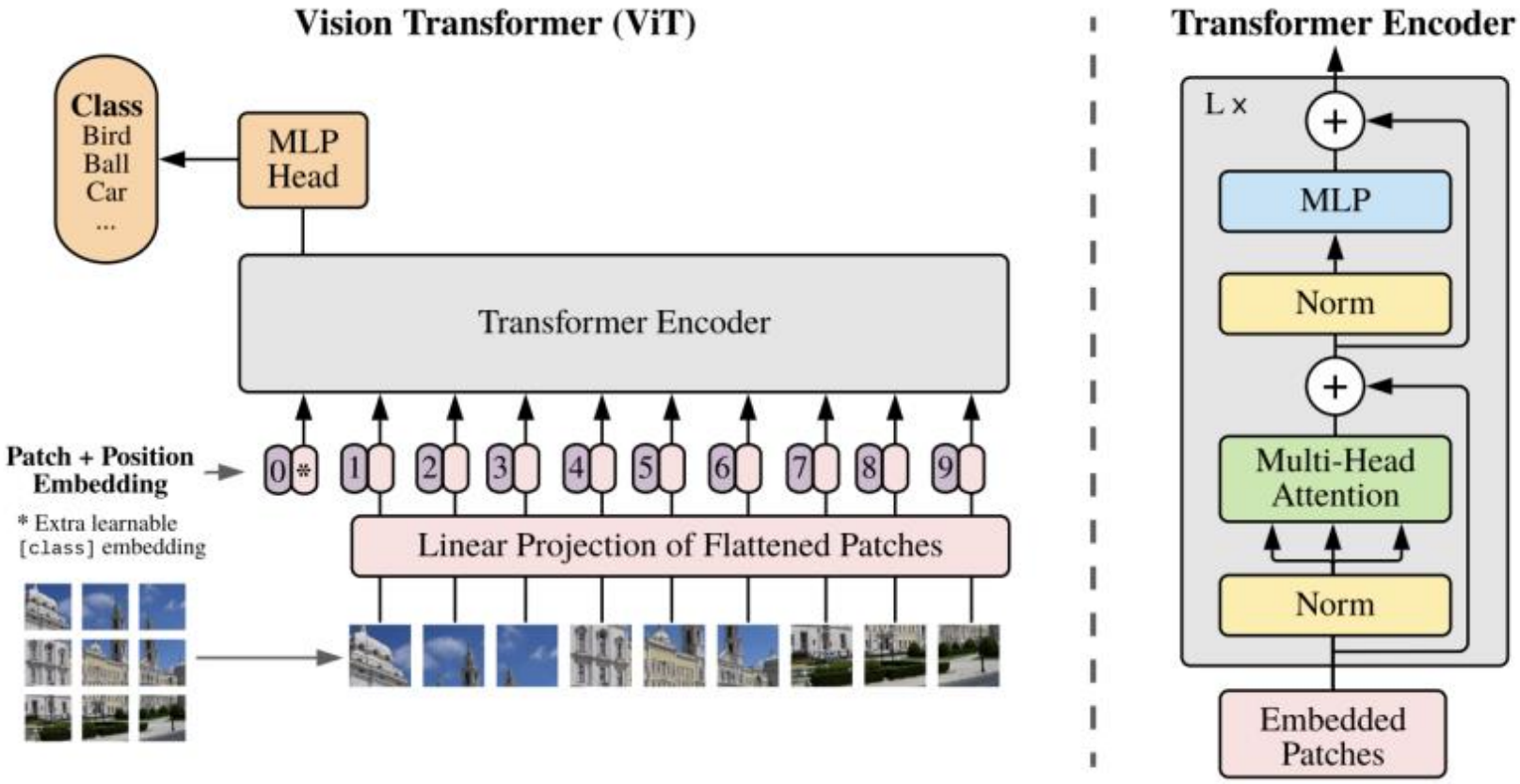
A woman holding a clock in her hand.

Spatial Attention

- Transformers have been successful in learning word representations in NLP. Can we extend this to computer vision?
  - Transformers are designed for sequence modeling. How can we convert a 2D RGB image into a sequence?

Vision Transformer (ViT)

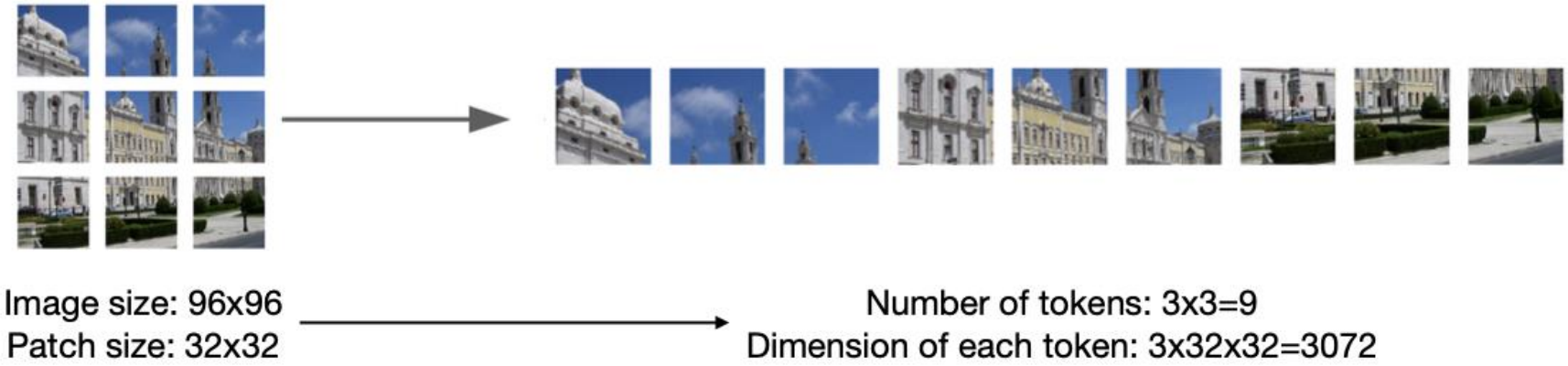
- Overview of the Vision Transformer (ViT)
  - The major difference of Transformers in NLP is how to prepare the input tokens to Transformer layers



Dosovitskiy, Alexey. "An image is worth 16x16 words: Transformers for image recognition at scale." International Conference on Learning Representations 2019.

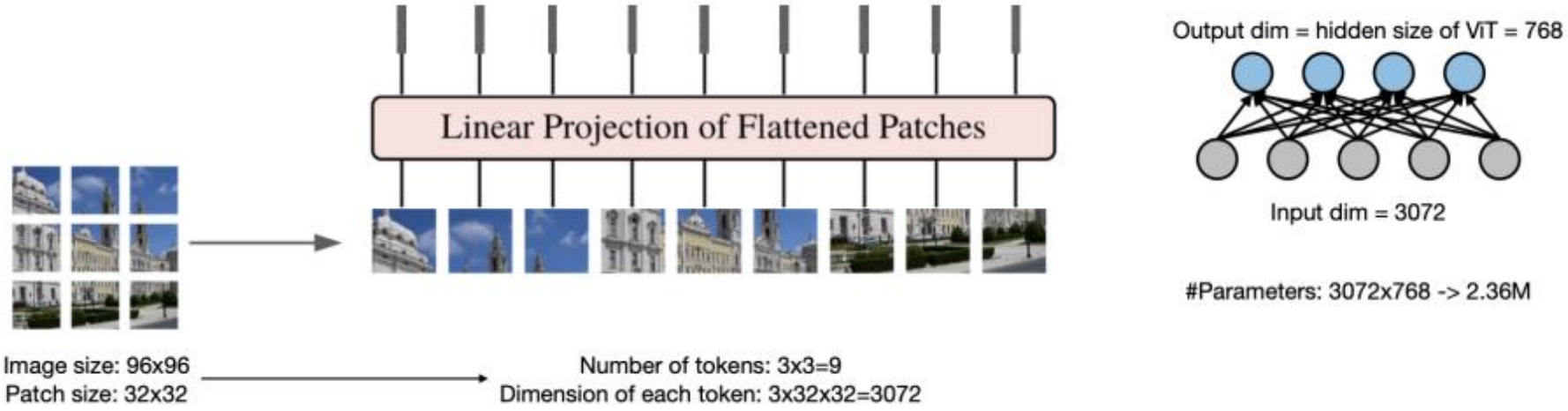
Tokenization of ViT

- The basic idea is dividing the image into a sequence of patches
  - Each patch will correspond to a token
  - The initial dimension of token could be the simple concatenation of all the elements in the patch



Tokenization of ViT

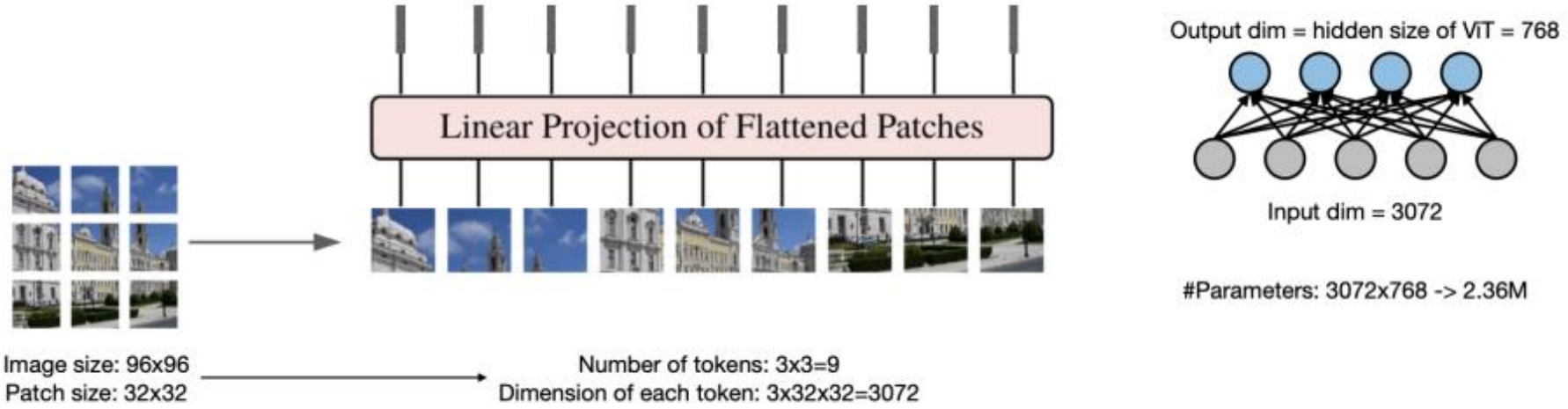
- The basic idea is dividing the image into a sequence of patches
  - Then we use several **linear layers** to project the initial tokens to the dimension of Transformer layers



Tokenization of ViT

- The basic idea is dividing the image into a sequence of patches
  - Then we use several **linear layers** to project the initial tokens to the dimension of Transformer layers

Tokenization of ViT is formulated as $f\colon \mathbb{R}^{H\times W} \to \mathbb{R}^{D}$, which converts each image patch to a vector (i.e., **patch embedding**).

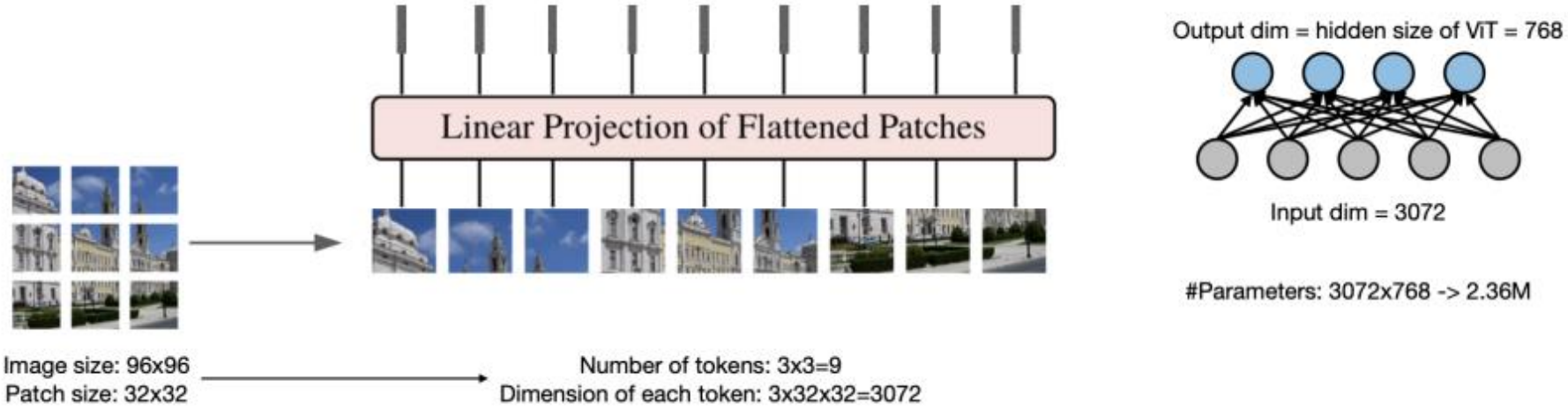


Tokenization of ViT

- The basic idea is dividing the image into a sequence of patches
  - Then we use several **linear layers** to project the initial tokens to the dimension of Transformer layers

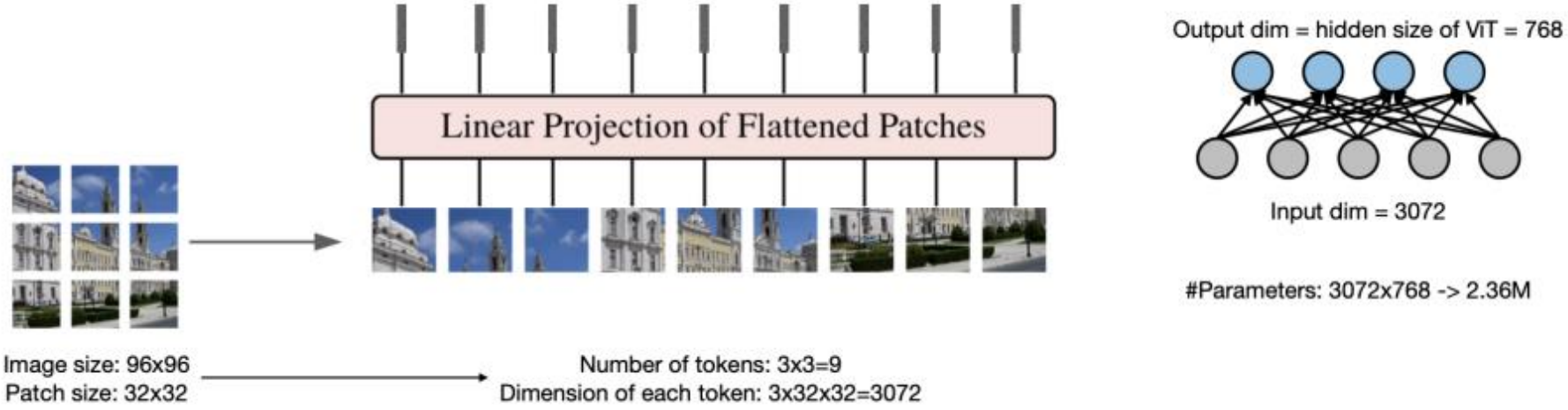
Tokenization of ViT is formulated as f: $\mathbb{R}^{H\times W} \rightarrow \mathbb{R}^{D}$, which converts each image patch to a vector (i.e., **patch embedding**).

Did we see such transformation before? **Image classification!**



Tokenization of ViT

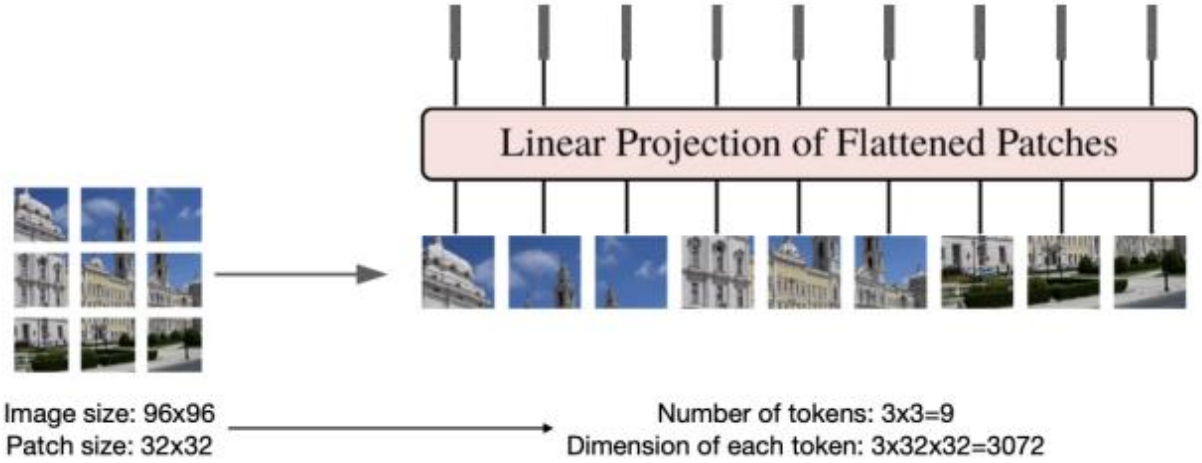
- Recall: We used to flatten the image pixels and use linear layers to train an image classification model
  - This is exactly what we are doing here for patch embedding

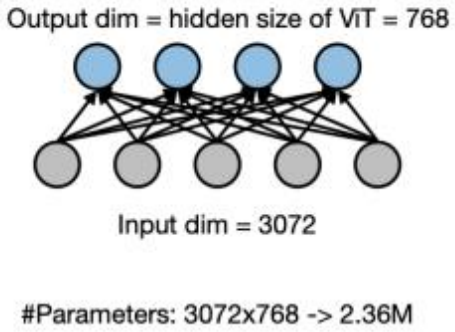


Tokenization of ViT

- Recall: We used to flatten the image pixels and use linear layers to train an image classification model
  - This is exactly what we are doing here for patch embedding

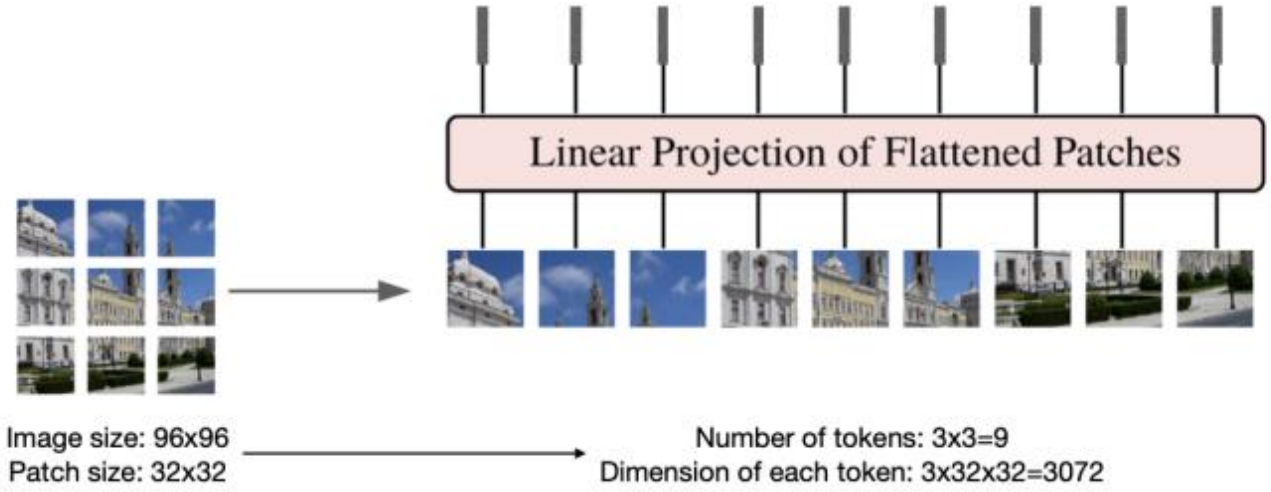
Since linear layer works, CNN should also work.





Tokenization of ViT

- In practice, we use convolution layers for linear projection
  - Compared to linear layers, convolution captures spatial information and forms better patch embedding



The parameters of convolution depend on the patch size

```
32x32 Conv, stride 32, padding 0
in_channels=3, out_channels = 768
```

Quiz

For an input RGB image of size 224×224 with patch size of 16×16. The hidden size of Transformer layer is 768. How should we configure the linear projection layer for patch embedding?

Q1: Use linear layers. Which of the following statement is NOT correct?

A The number of tokens is 196

B The input channel of the linear layer is 256

C The output channel of the linear layer is 768

Quiz

For an input RGB image of size 224×224 with patch size of 16×16. The hidden size of Transformer layer is 768. How should we configure the linear projection layer for patch embedding?

Q1: Use linear layers. Which of the following statement is NOT correct?

A The number of tokens is 196

B The input channel of the linear layer is 256

C The output channel of the linear layer is 768

Correct Answer: B

The input channel should be 16 * 16 * 3 = 768

Quiz

For an input RGB image of size 224×224 with patch size of 16×16. The hidden size of Transformer layer is 768. How should we configure the linear projection layer for patch embedding?

Q2: Use convolution layers. No overlapping among patches. Which of the following statement is NOT correct?

A The number of tokens is 196

B The input channel of the convolution layer is 3

C The kernel size of the convolution layer is 16

D The stride of the convolution layer is 1

Quiz

For an input RGB image of size 224×224 with patch size of 16×16. The hidden size of Transformer layer is 768. How should we configure the linear projection layer for patch embedding?

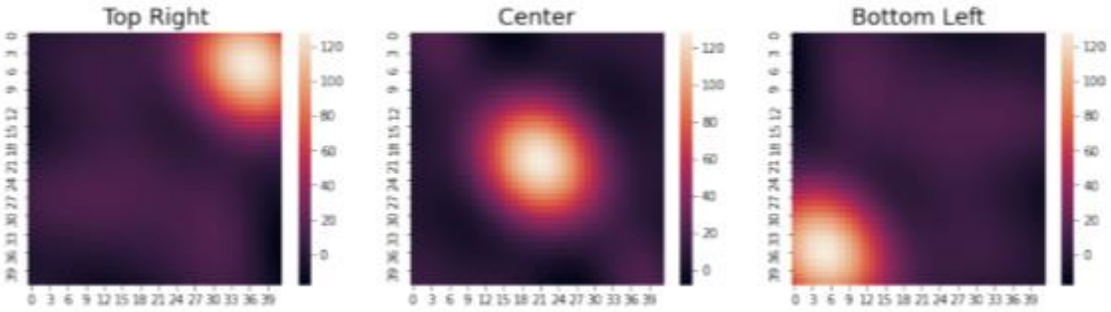
Q2: Use convolution layers. No overlapping among patches. Which of the following statement is NOT correct?

A The number of tokens is 196
B The input channel of the convolution layer is 3
C The kernel size of the convolution layer is 16
D The stride of the convolution layer is 1

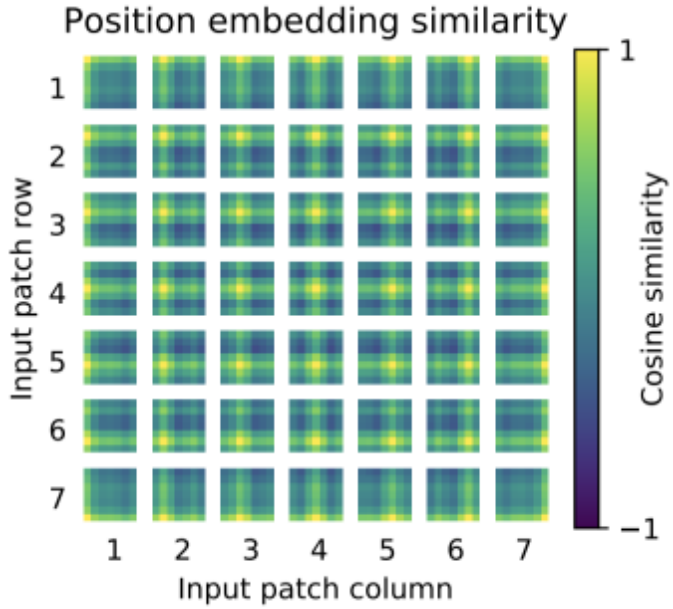
Correct Answer: D
The stride should be 16 for non-overlapping patch embedding

Positional Encoding

- Like positional embeddings in NLP, the positional embeddings in ViT ensure that spatially closer tokens have higher dot-product similarities.

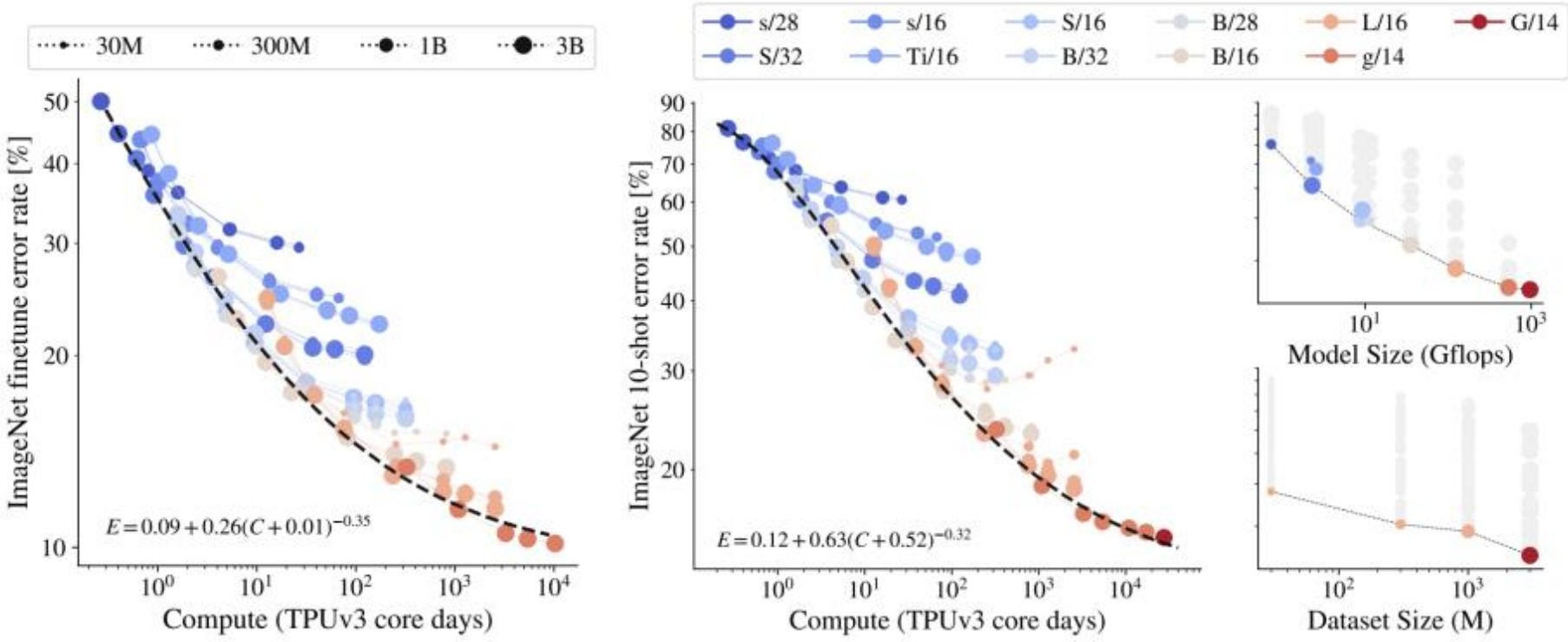


Similarity maps of pixels at different locations



Scaling Law of Transformer

- The impressive scaling laws of Transformers motivate the development of large models.
  - Given large model size and vast amounts of data, Transformer shows incredible performance and generalization
  - These large-scale pre-trained models are called **foundation model**

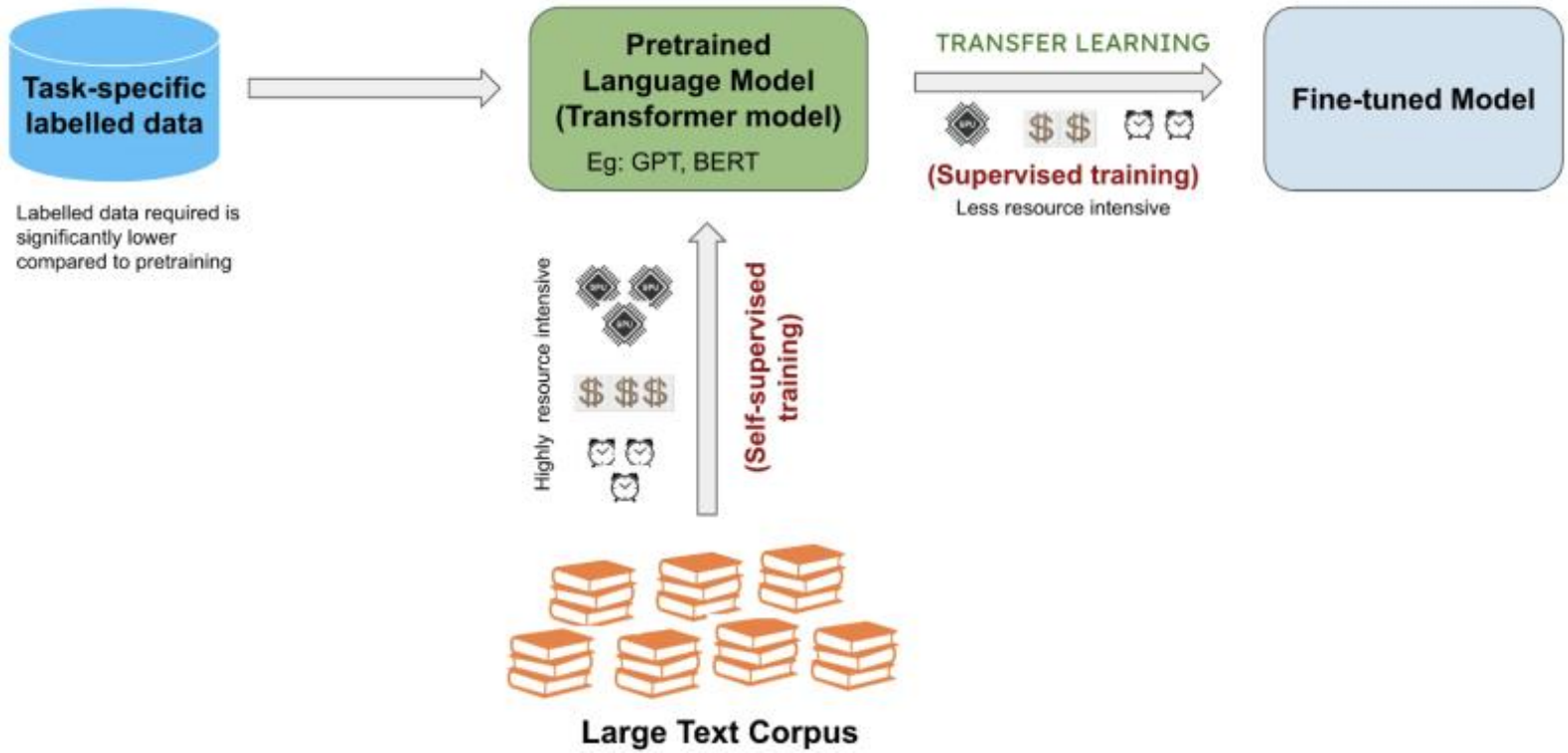




Part 3. Variants of Transformers

Pretraining and Finetuning

- The pretraining and finetuning paradigm.
  - Pretrain a foundation model on large unlabeled data
  - Finetune with small labeled data on a specific downstream task



Parameter-efficient fine-tuning: Adapter

- When you download a large-scale pretrained Transformer online, it‘s likely that the model won’t perform well on your specific tasks
  - E.g., an ImageNet-pretrained ViT may not work for tumor segmentation from CTs

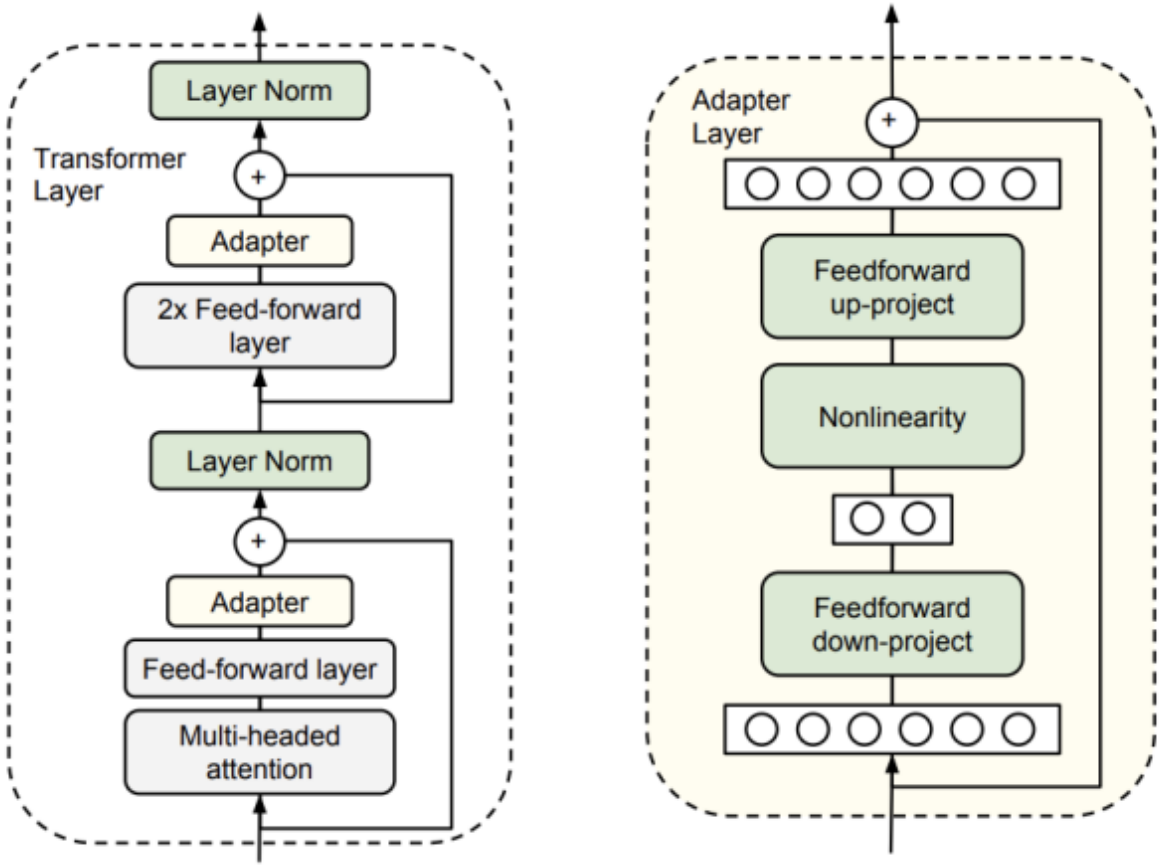
Parameter-efficient fine-tuning: Adapter

- When you download a large-scale pretrained Transformer online, it‘s likely that the model won’t perform well on your specific tasks
  - E.g., an ImageNet-pretrained ViT may not work for tumor segmentation from CTs

- You might want to fine-tune the Transformer on your data, but aim to train as few parameters as possible for some reasons:
  - Training a large Transformer model is of high cost
  - Your data for the specific task is usually limited for training a large Transformer, leading to overfitting issues.

Parameter-efficient fine-tuning: Adapter

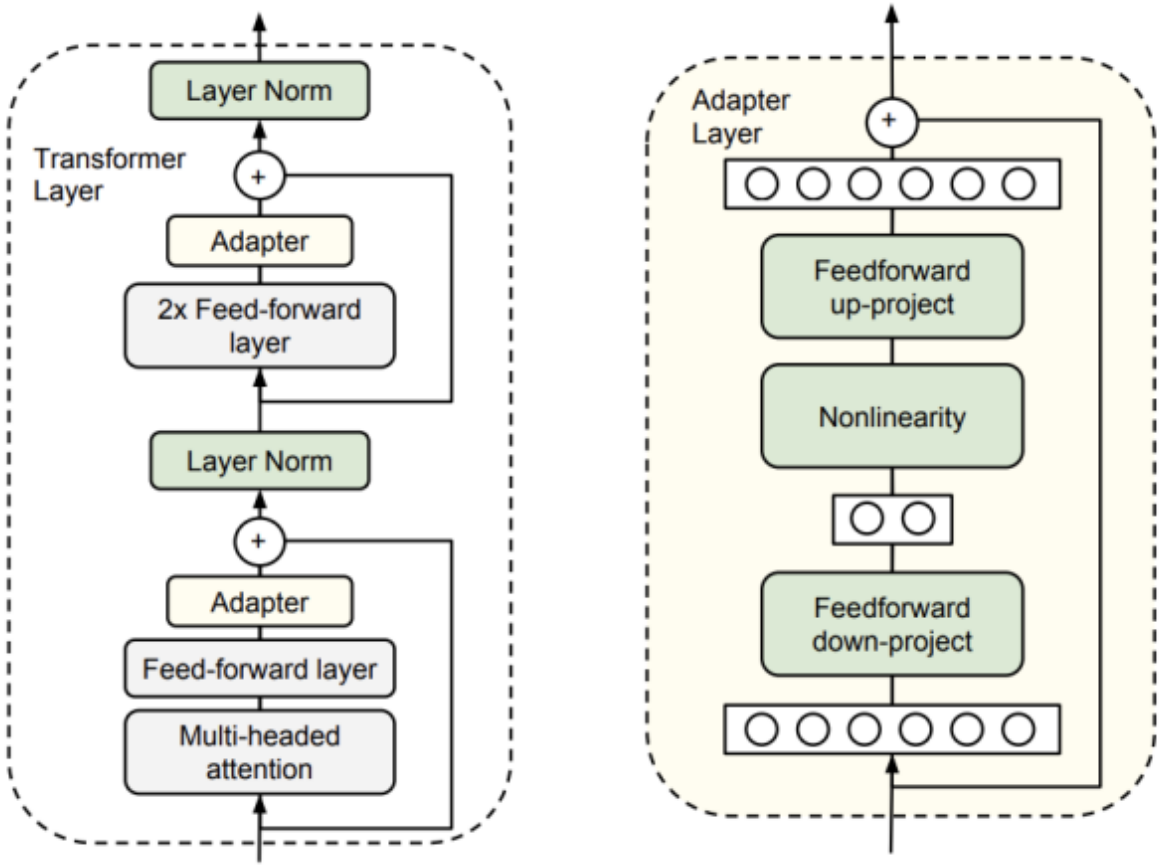
- Adapter, composed of linear layers and nonlinear activation functions, is inserted after attention and feed-forward layers in Transformer blocks.



Only **green modules** are trained in the adaptation

Parameter-efficient fine-tuning: Adapter

- The parameters of adapter layers are significantly fewer than those of the original Transformer (usually less than 2%).
- Training only the adapter makes the fine-tuning process highly efficient.



Only **green modules** are trained in the adaptation

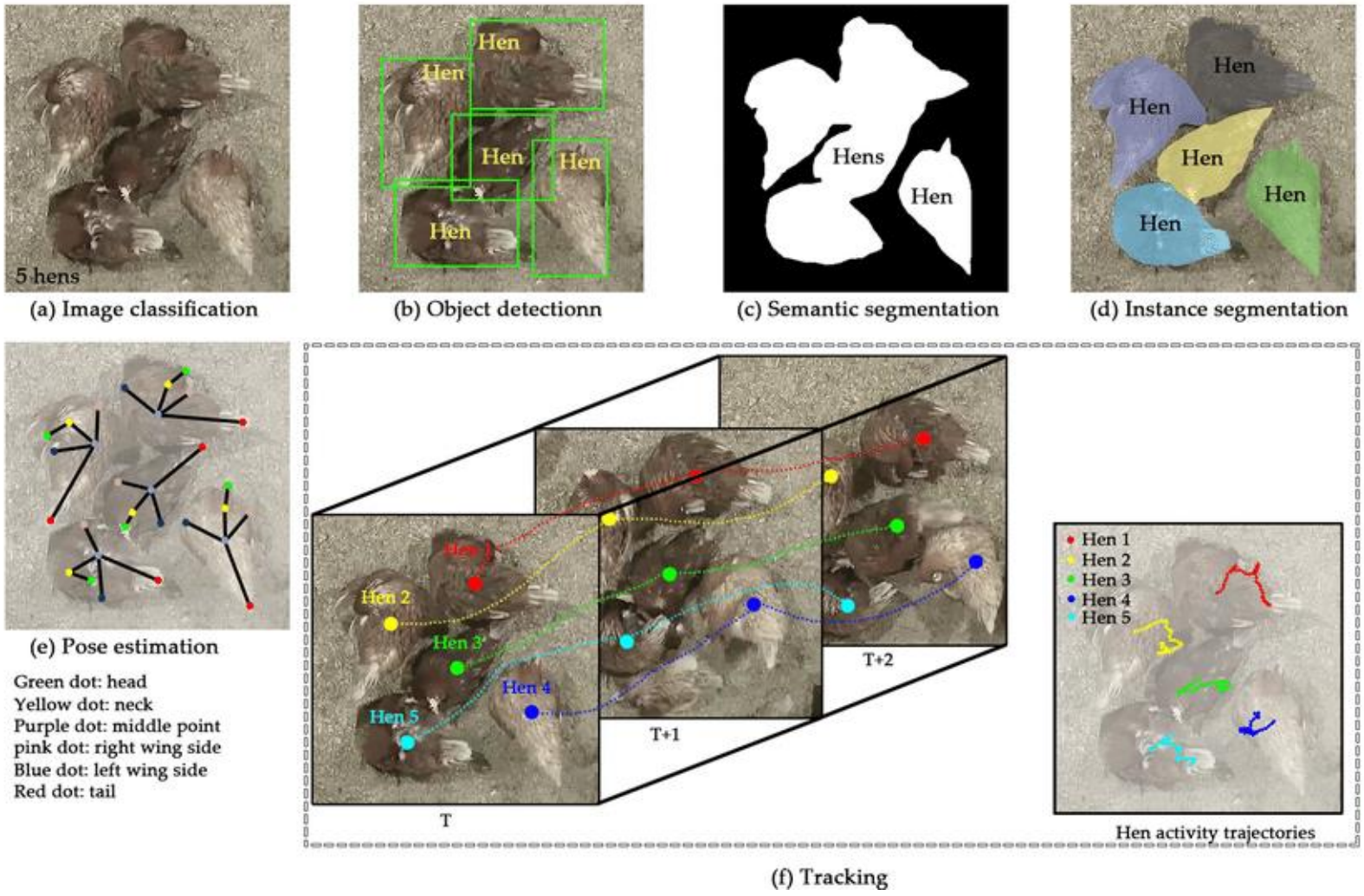
Parameter-efficient fine-tuning: Adapter

- Experiment results: multiple natural language classification datasets
  - Even when training a small proportion of the parameters, it achieves performance comparable to training all the parameters.

Dataset	No BERT baseline	$\text{BERT}_{\text{BASE}}$ Fine-tune	$\text{BERT}_{\text{BASE}}$ Variable FT	$\text{BERT}_{\text{BASE}}$ Adapters
20 newsgroups	91.1	92.8 ± 0.1	92.8 ± 0.1	91.7 ± 0.2
Crowdflower airline	84.5	83.6 ± 0.3	84.0 ± 0.1	84.5 ± 0.2
Crowdflower corporate messaging	91.9	92.5 ± 0.5	92.4 ± 0.6	92.9 ± 0.3
Crowdflower disasters	84.9	85.3 ± 0.4	85.3 ± 0.4	84.1 ± 0.2
Crowdflower economic news relevance	81.1	82.1 ± 0.0	78.9 ± 2.8	82.5 ± 0.3
Crowdflower emotion	36.3	38.4 ± 0.1	37.6 ± 0.2	38.7 ± 0.1
Crowdflower global warming	82.7	84.2 ± 0.4	81.9 ± 0.2	82.7 ± 0.3
Crowdflower political audience	81.0	80.9 ± 0.3	80.7 ± 0.8	79.0 ± 0.5
Crowdflower political bias	76.8	75.2 ± 0.9	76.5 ± 0.4	75.9 ± 0.3
Crowdflower political message	43.8	38.9 ± 0.6	44.9 ± 0.6	44.1 ± 0.2
Crowdflower primary emotions	33.5	36.9 ± 1.6	38.2 ± 1.0	33.9 ± 1.4
Crowdflower progressive opinion	70.6	71.6 ± 0.5	75.9 ± 1.3	71.7 ± 1.1
Crowdflower progressive stance	54.3	63.8 ± 1.0	61.5 ± 1.3	60.6 ± 1.4
Crowdflower US economic performance	75.6	75.3 ± 0.1	76.5 ± 0.4	77.3 ± 0.1
Customer complaint database	54.5	55.9 ± 0.1	56.4 ± 0.1	55.4 ± 0.1
News aggregator dataset	95.2	96.3 ± 0.0	96.5 ± 0.0	96.2 ± 0.0
SMS spam collection	98.5	99.3 ± 0.2	99.3 ± 0.2	95.1 ± 2.2
Average	72.7	73.7	74.0	73.3
Total number of params	—	17×	9.9×	1.19×
Trained params/task	—	100%	52.9%	1.14%

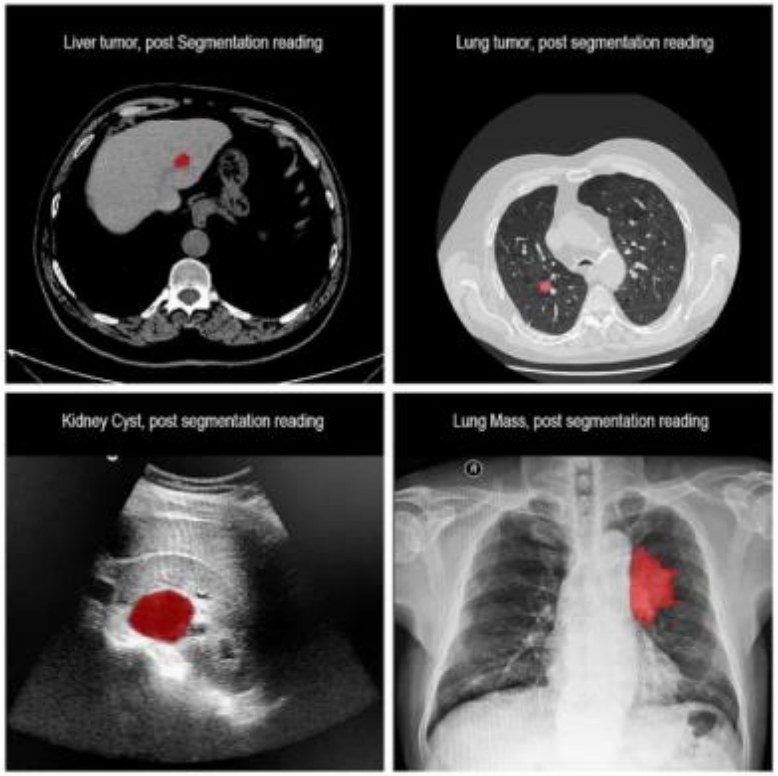
Downstream Tasks for ViT

- Large pretrained ViTs can be transferred to various downstream tasks through fine-tuning.



ViT for Dense Prediction

- ViT finds it challenging to handle dense prediction tasks such as segmentation, object detection, depth estimation due to the high computation cost of self-attention.

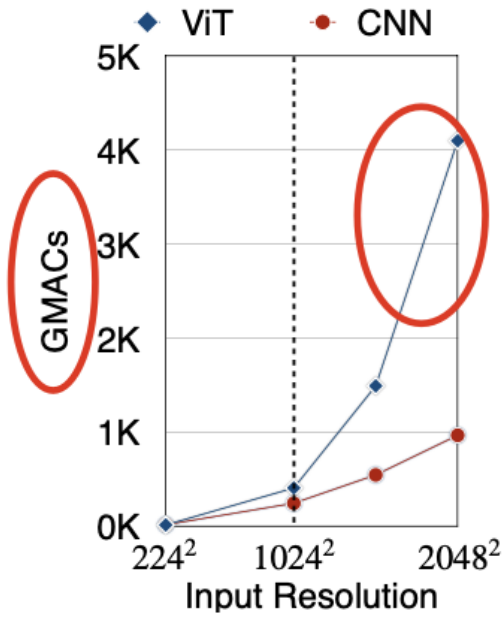




Computation Complexity of Self-Attention

- ViTs have **quadratic computation complexity to input image size** due to computation of self-attention globally
  - The computational cost of a global multi-head self-attention module with hidden channel C of an image of $h \times w$ patches is

Cost of Self-Attention: $4hwC^2 + 2(hw)^2C$



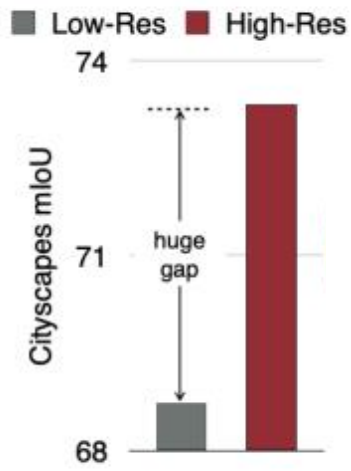
ViT for Dense Prediction

- On the one hand, patch encoding reduces the resolution of the image with a scale of patch size.
  - The information of original resolution is hard to recover for pixel-wise prediction.
- On the other hand, Its intractable for ViT on high-resolution images due to its quadratic computational complexity to image size.

Details and small objects are missing in low-resolution images.

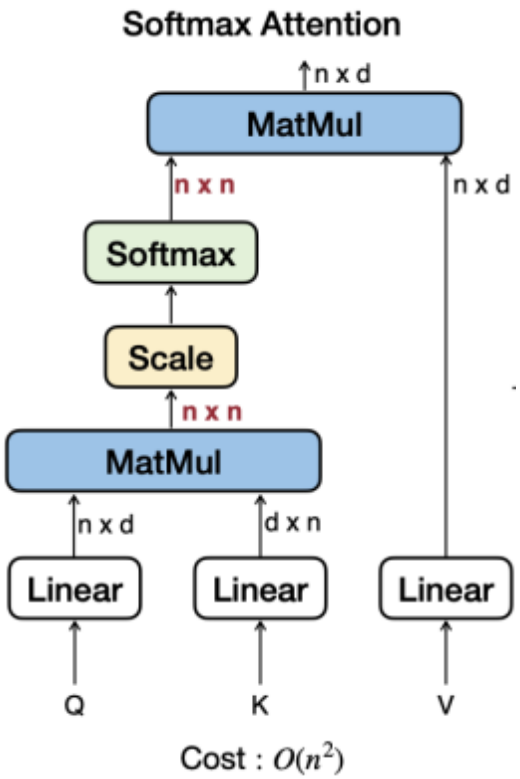




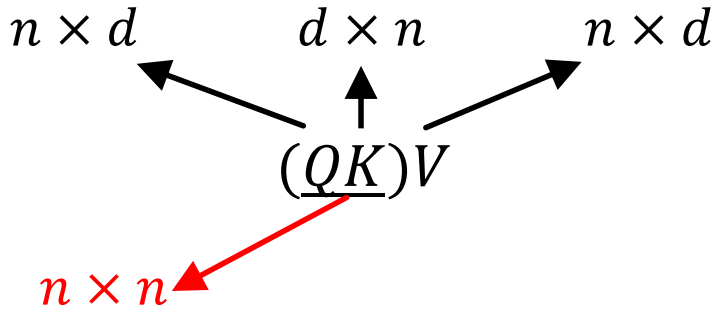


Linear Attention (EfficientViT)

- The quadratic complexity of attention comes from the maintenance and matrix manipulation of a $n \times n$ matrix.
- n is the number of tokens, d is the hidden dimension of ViT.



Can we avoid such matrix to finish the attention calculation?



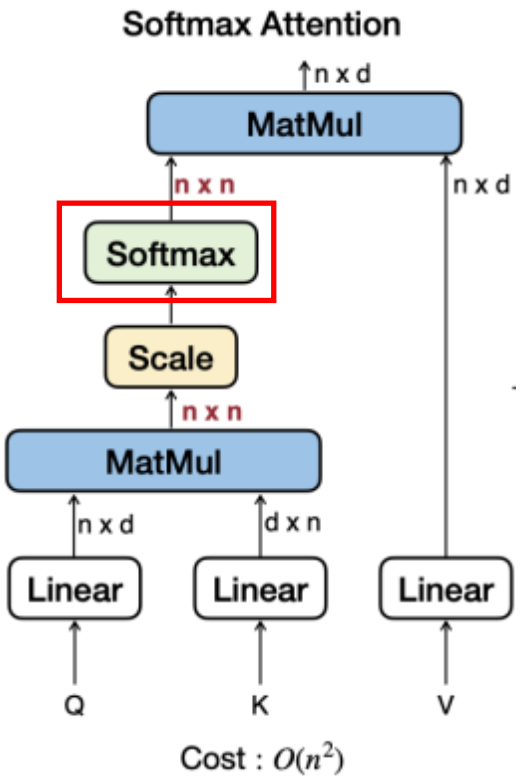
This is the criminal of high complexity.

We don't want this!

Cai, Han, et al. "Efficientvit: Multi-scale linear attention for high-resolution dense prediction." ICCV 2023

Linear Attention (EfficientViT)

- The quadratic complexity of attention comes from the maintenance and matrix manipulation of a $n \times n$ matrix



Can we avoid such matrix to finish the attention calculation?

$n \times d$ $\quad$ $d \times n$ $\quad$ $n \times d$

$(\underline{QK})V$

Associate property of matrix manipulation: $(AB)C = A(BC)$

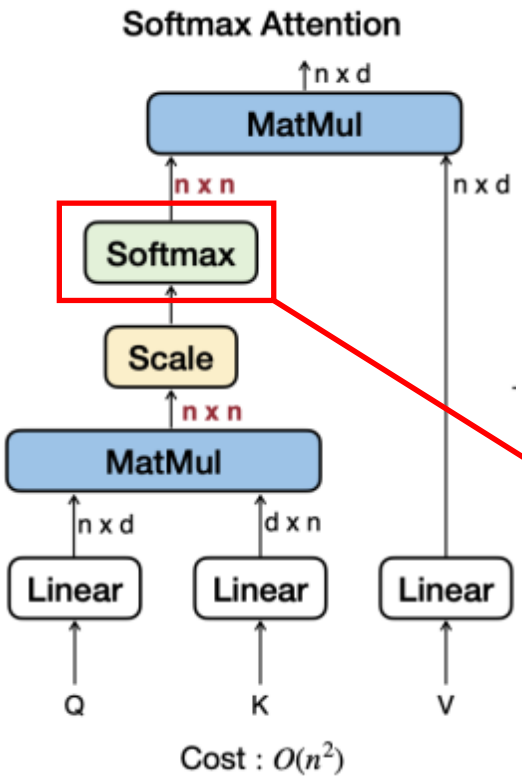
What if $\quad Q(\underline{KV})$

$d \times d$

Cai, Han, et al. "Efficientvit: Multi-scale linear attention for high-resolution dense prediction." ICCV 2023

Linear Attention (EfficientViT)

- The quadratic complexity of attention comes from the maintenance and matrix manipulation of a $n \times n$ matrix



Can we avoid such matrix to finish the attention calculation?

$n \times d$ $\quad$ $d \times n$ $\quad$ $n \times d$

$(\underline{QK})V$

Associate property of matrix manipulation: $(AB)C = A(BC)$

What if $\quad Q(\underline{KV})$

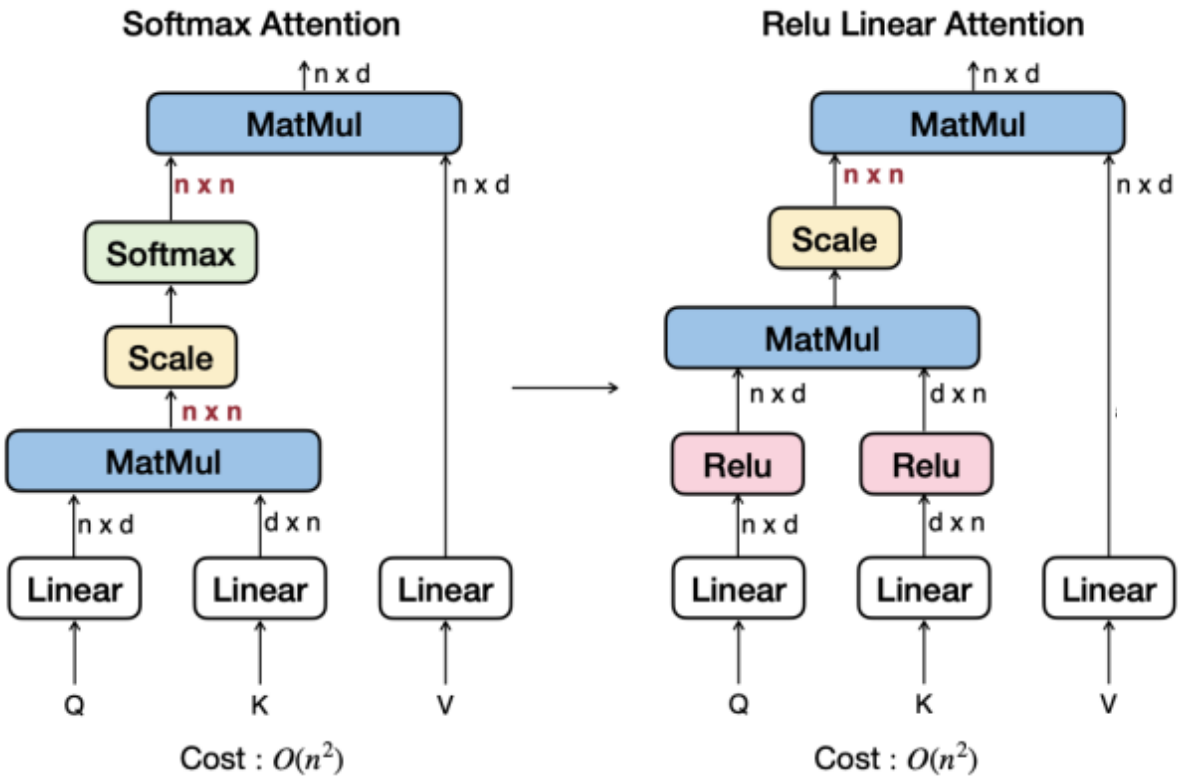
$d \times d$

BUT! There is non-linearity in attention. We can't just simply apply the associate property.

Cai, Han, et al. "Efficientvit: Multi-scale linear attention for high-resolution dense prediction." ICCV 2023

Linear Attention (EfficientViT)

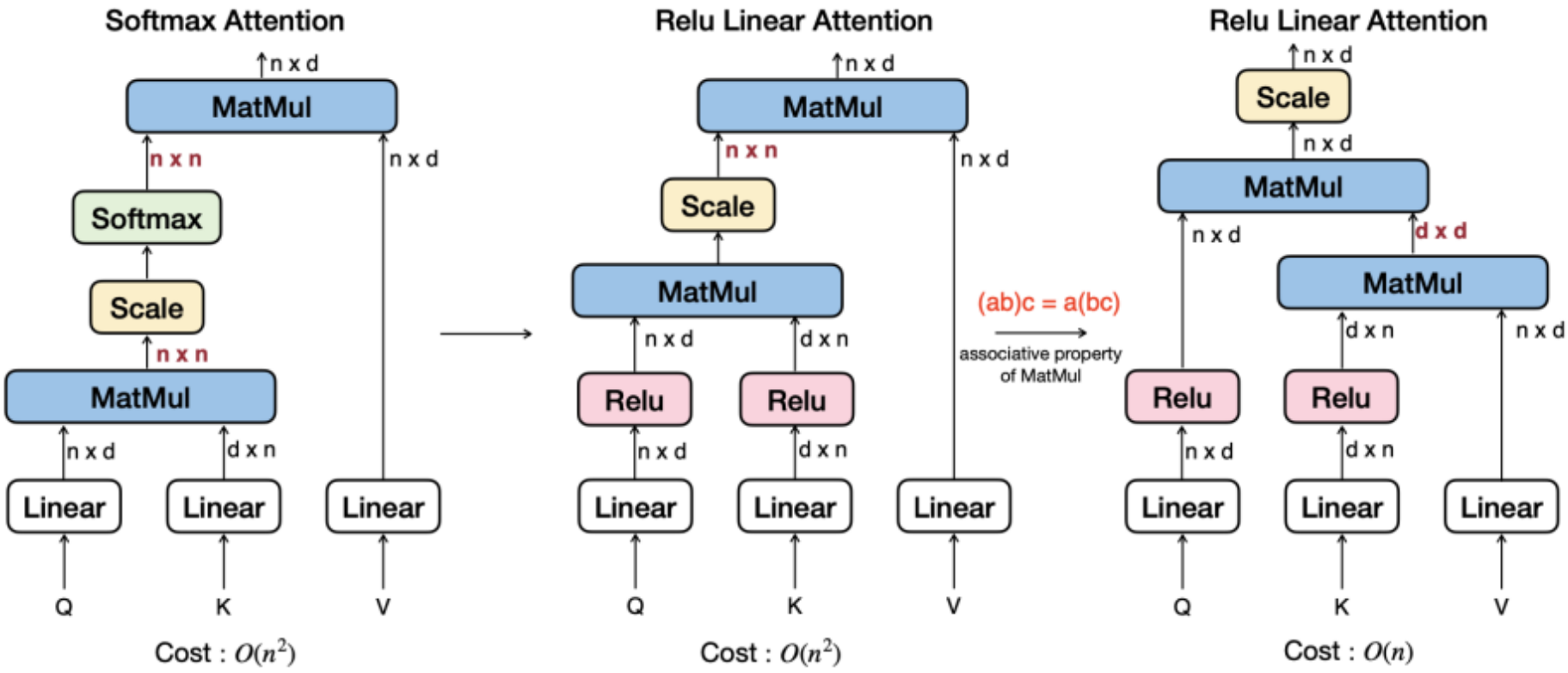
- Therefore, the first step is to replace the non-linear Softmax in the attention with linear MatMul.
  - The non-linearity is added by ReLU before the attention calculation, which won't affect the associate property.



Cai, Han, et al. "Efficientvit: Multi-scale linear attention for high-resolution dense prediction." ICCV 2023

Linear Attention (EfficientViT)

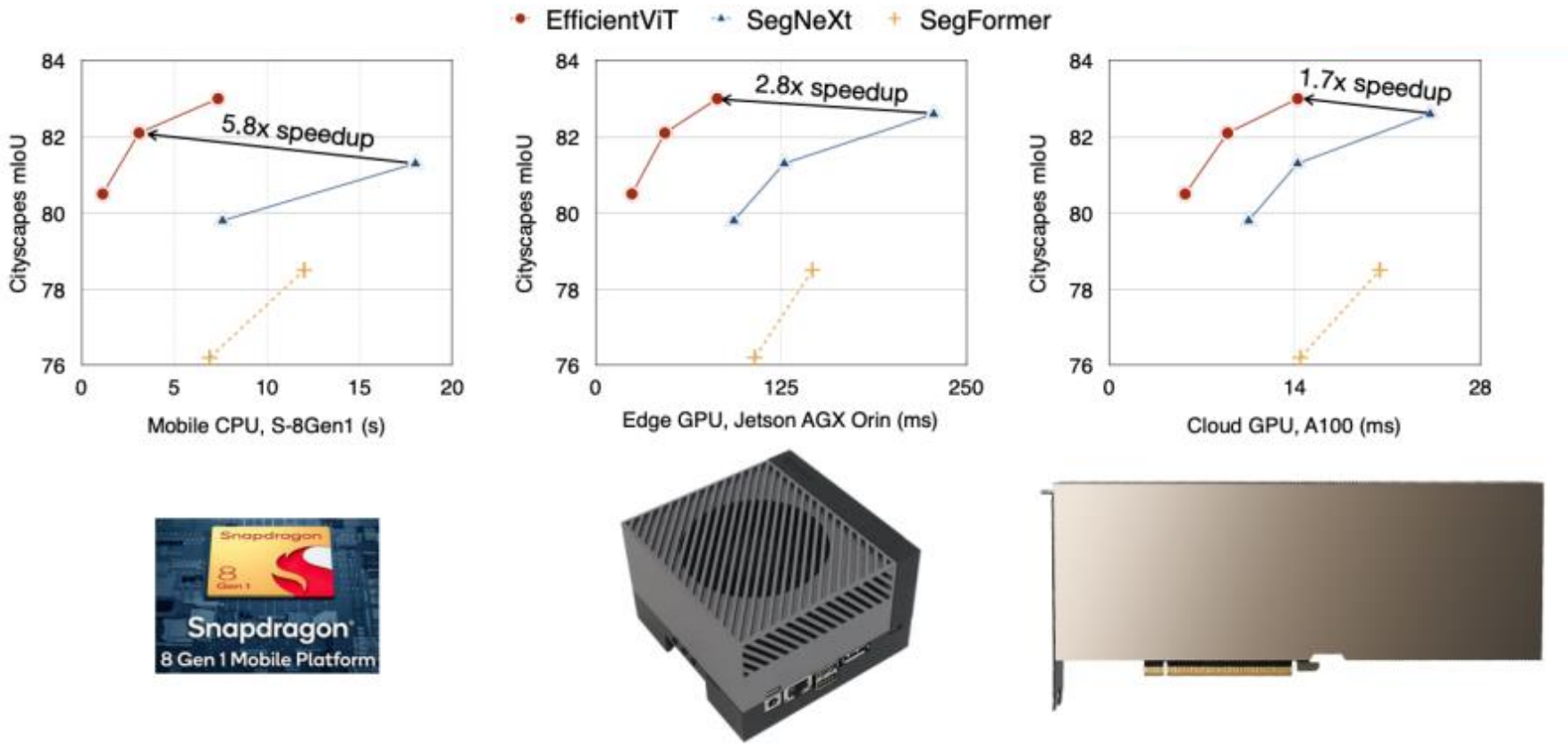
- Then we can apply the associate property, and get linear attention with complexity of $O(n)$



Cai, Han, et al. “Efficientvit: Multi-scale linear attention for high-resolution dense prediction.” ICCV 2023

Linear Attention (EfficientViT)

- It provides consistent and significant speedup on diverse hardware platforms.

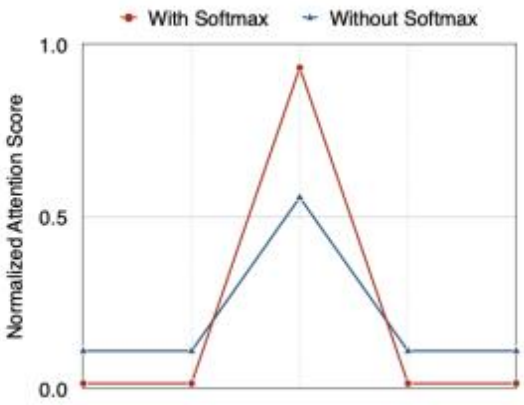


Cai, Han, et al. “Efficientvit: Multi-scale linear attention for high-resolution dense prediction.” ICCV 2023

Linear Attention (EfficientViT)

- However, linear attention cannot produce sharp distributions, which may sacrifice performance.
- Thus, it is good at capturing global context information but bad at capturing local information.





Cai, Han, et al. “Efficientvit: Multi-scale linear attention for high-resolution dense prediction.” ICCV 2023

Linear Attention (EfficientViT)

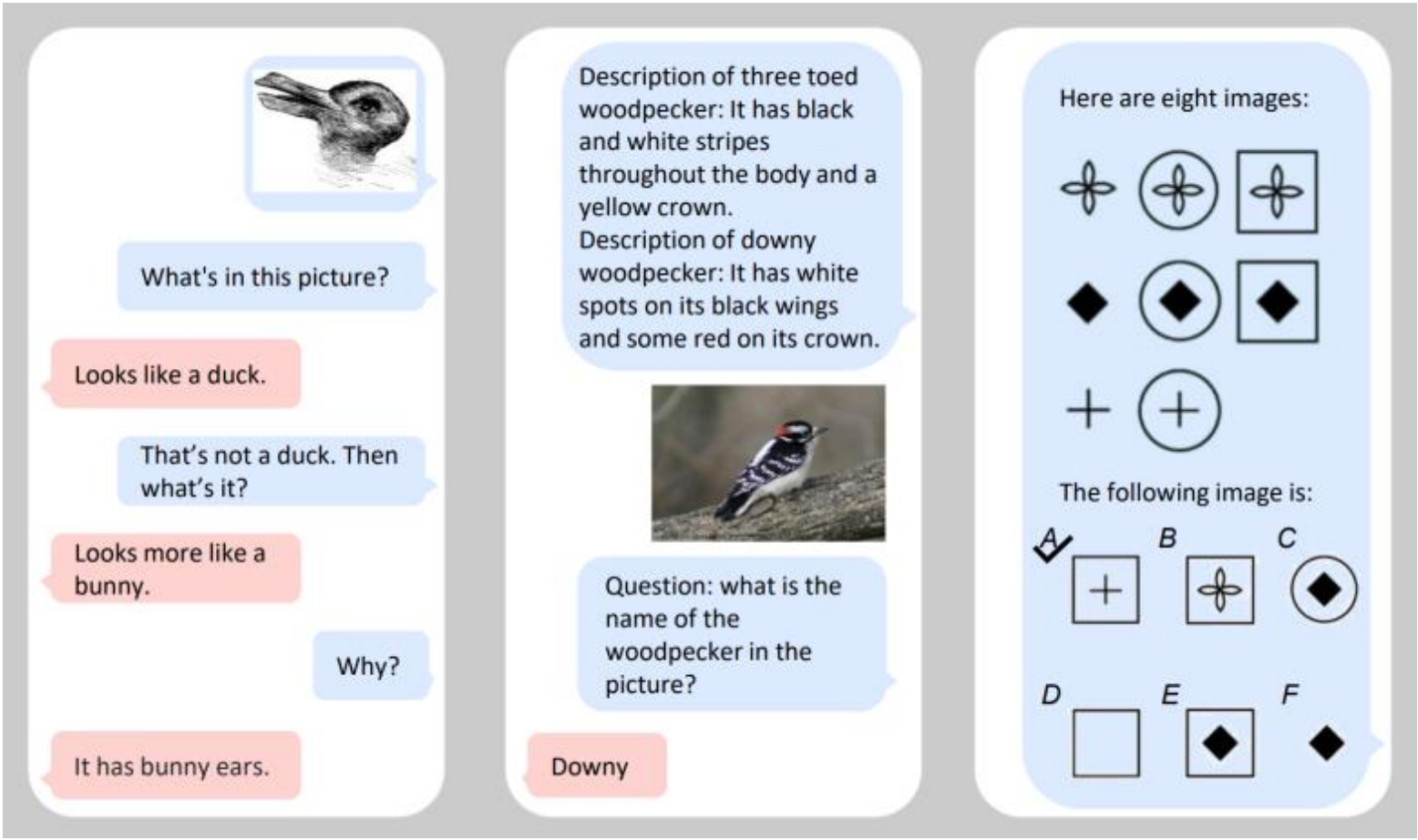
- Results of Segment Anything with EfficientViT as the backbone.



Cai, Han, et al. “Efficientvit: Multi-scale linear attention for high-resolution dense prediction.” ICCV 2023

Multimodal Transformer

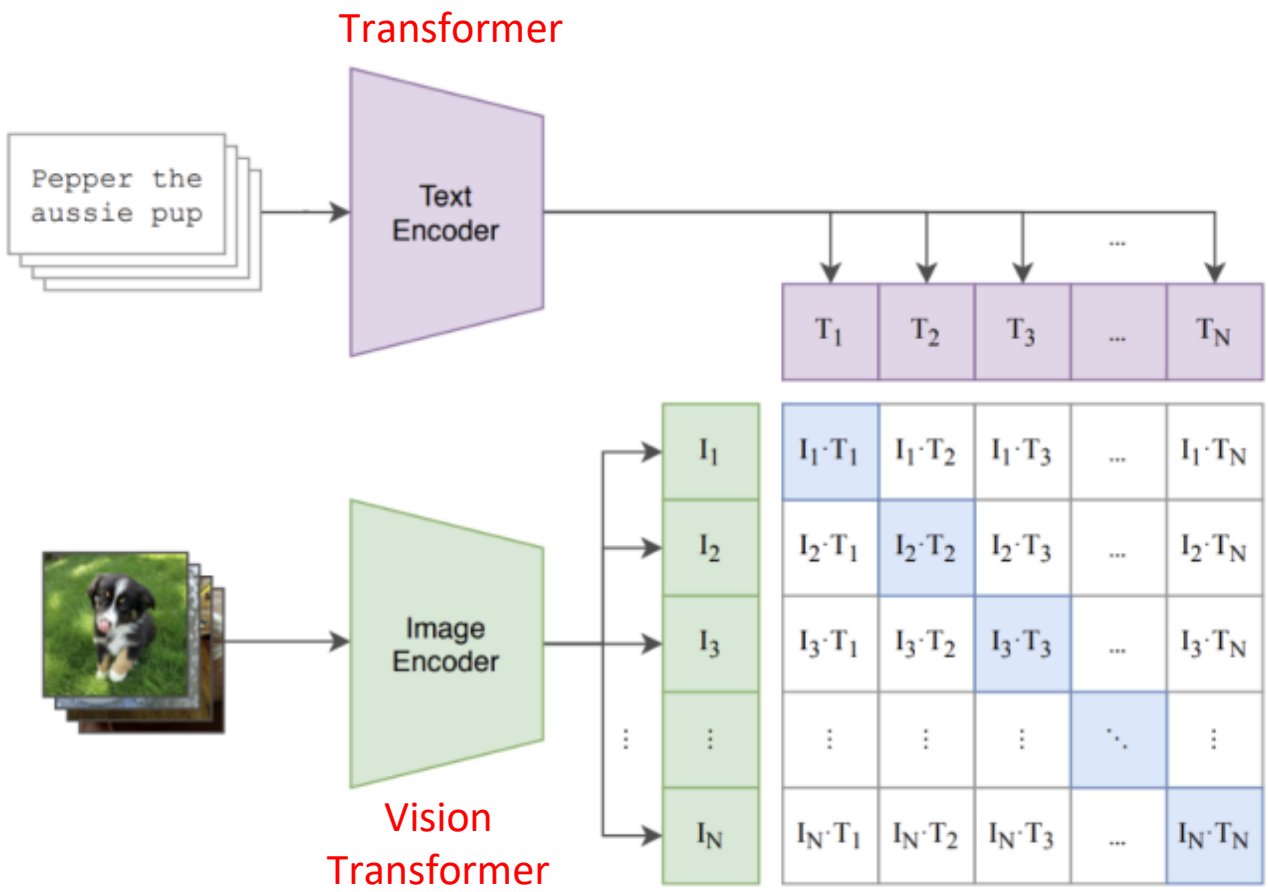
- As a universal architecture for both NLP and CV, transformer acts as a bridge connecting multiple modalities for collaboration.



Applications of multimodal LLMs using transformers as the architecture

Multimodal Transformer: CLIP

- CLIP (Contrastive Language-Image Pre-Training) models have two transformer branches: a vision branch and a text branch
  - The two branches encode the an image/sentence into a vector



Multimodal Transformer: CLIP

- CLIP is trained with text supervision by contrastive learning
  - The training dataset consists of image-text pairs, with each image accompanied by a descriptive text.



"motorcycle front wheel"



"*thumbnail for version as of 21 57 29 june 2010*"



"file frankfurt airport skyline 2017 05 jpg"



"file london barge race 2 jpg"



"moustache seamless wallpaper design"

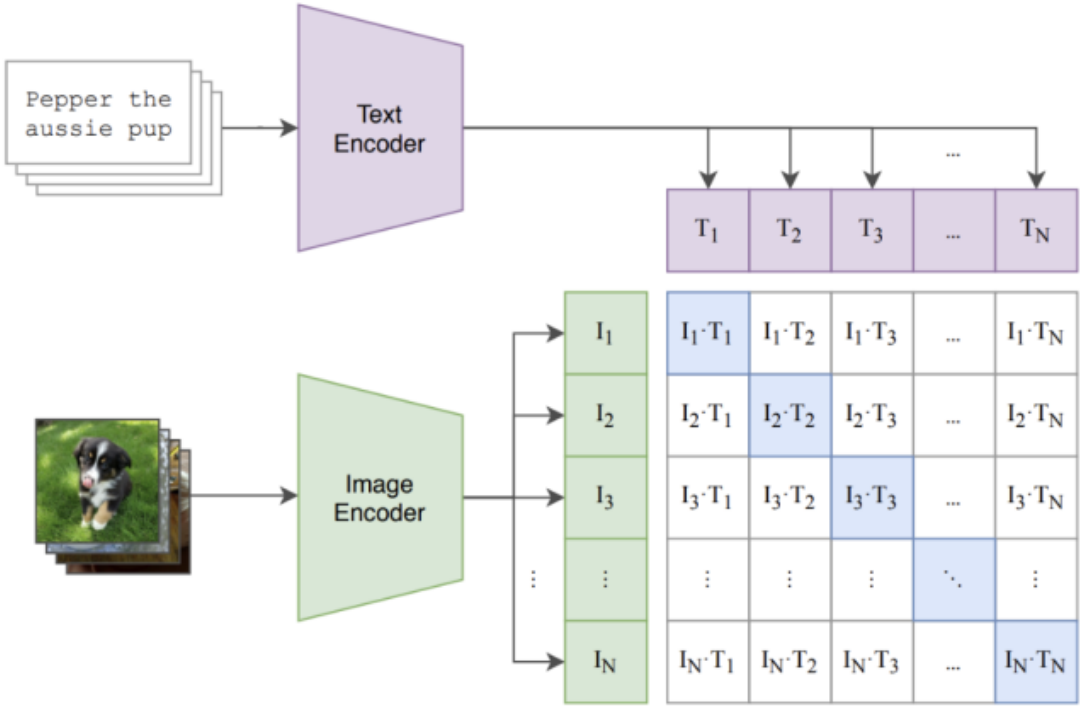


"st oswalds way and shops"

Multimodal Transformer: CLIP

- CLIP is trained with text supervision by contrastive learning
  - The training dataset consists of image-text pairs, with each image accompanied by a descriptive text.
  - The training brings the encoded image embedding closer to its paired text embedding, and pushing it away from other text embeddings (This can be achieved with the cross-entropy loss)

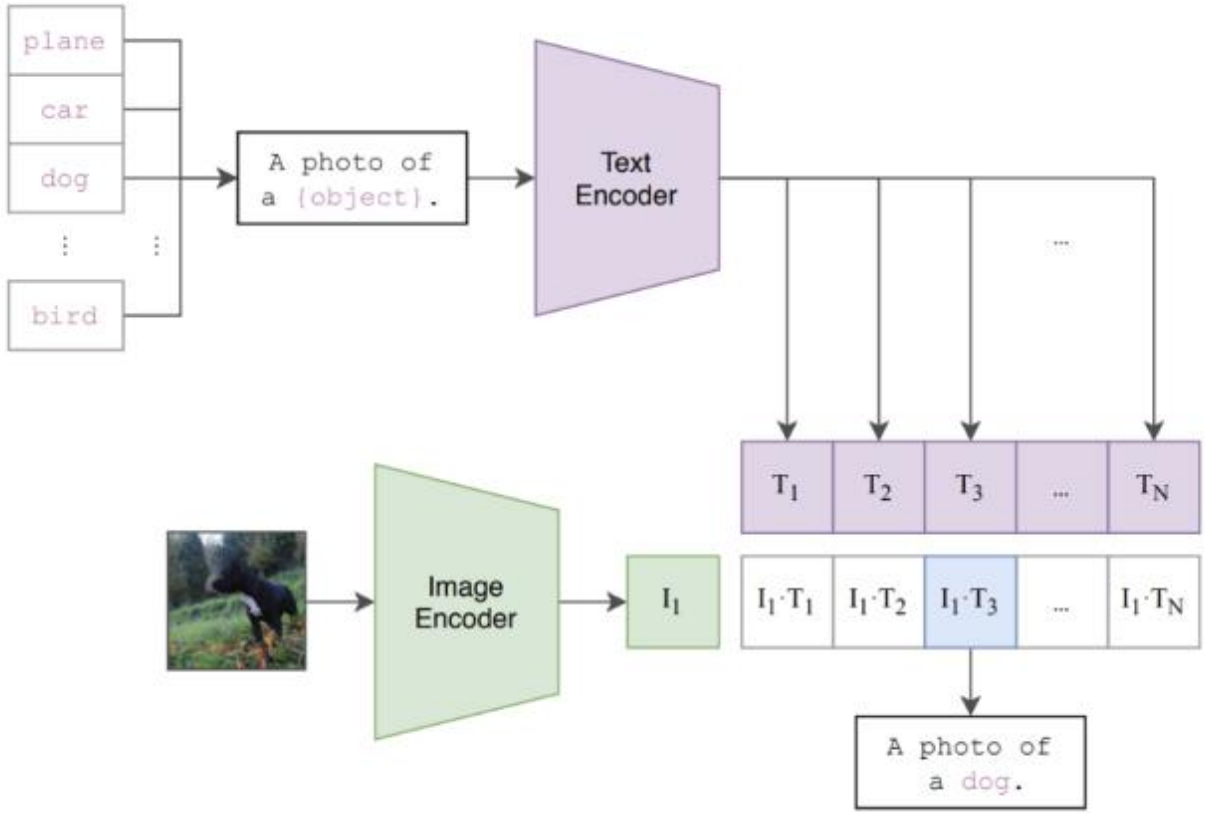
Training:



Multimodal Transformer: CLIP

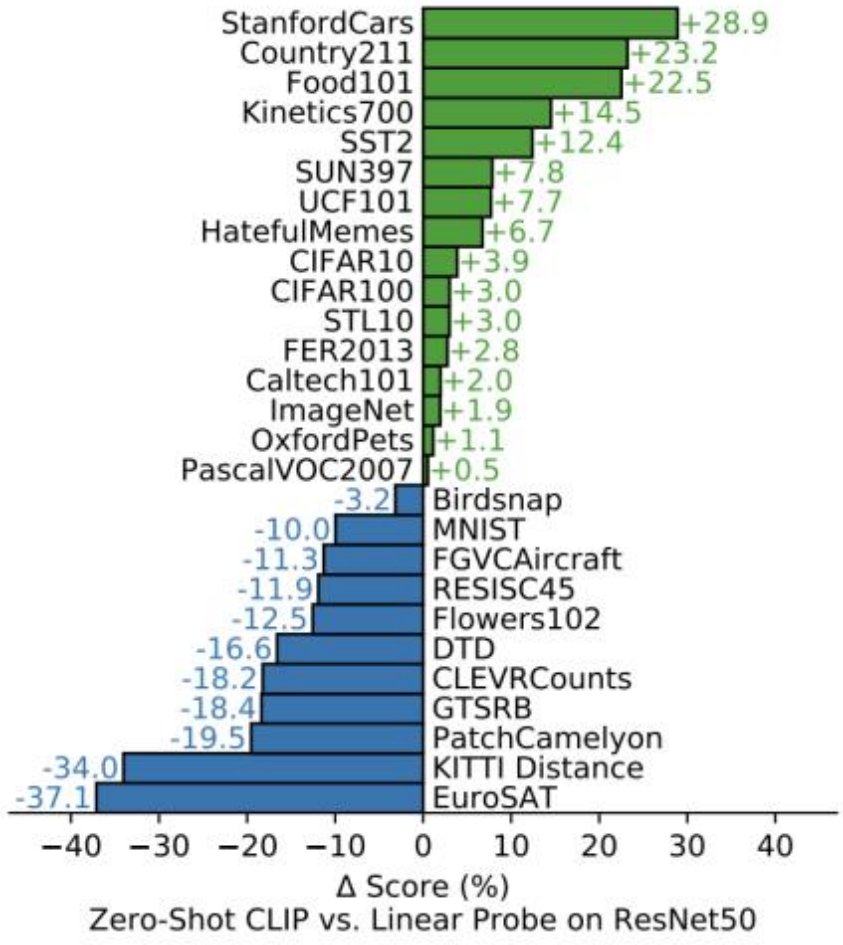
- CLIP enables zero-shot classification without any finetuning
  - Prepare a text description for each class
  - Do classification by matching the image to be classified with the text of all classes

Inference:



Multimodal Transformer: CLIP

- Zero-shot CLIP is competitive with a fully supervised baseline.
  - Across a 27 dataset eval suite, a zero-shot CLIP classifier outperforms a fully supervised classifier on 16 datasets



Compare zero-shot classification performance and supervised performance on multiple datasets



Part 4. Summary

Summary

- Basics of Transformer
  - Tokenization
  - Multi-head self-attention
  - Feed-forward
  - Positional encoding
- Vision Transformers
- Modern Transformers
  - Pre-training and Fine-tuning
  - Parameter-efficient fine-tuning
  - Efficient ViT for dense prediction: linear attention
  - Multimodal transformer: CLIP

Reading Materials

- Blog of transformer famliy: https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/
- Attention Mechanisms in Computer Vision: A Survey: https://arxiv.org/pdf/2111.07624
- A Cookbook of Self-Supervised Learning: https://arxiv.org/pdf/2304.12210
- Stanford CS25: Transformers Unite: https://web.stanford.edu/class/cs25/